

如果一个协议提供了这样的确保数据交付服务，就认为提供了**可靠数据传输**（reliable data transfer）。运输层协议能够潜在地向应用程序提供的一个重要服务是进程到进程的可靠数据传输。当一个运输协议提供这种服务时，发送进程只要将其数据传递进套接字，就可以完全相信该数据将能无差错地到达接收进程。

当一个运输层协议不提供可靠数据传输时，由发送进程发送的某些数据可能不能够到达接收进程。这可能会被**容忍丢失的应用**（loss-tolerant application）所接受，最值得注意的是多媒体应用，如交谈式音频/视频，它们能够承受一定量的数据丢失。在这些多媒体应用中，丢失的数据引起播放的音频/视频出现小干扰，而不是致命的损伤。

2. 吞吐量

在第1章中我们引入了可用吞吐量的概念，在沿着一条网络路径上的两个进程之间的通信会话场景中，可用吞吐量就是发送进程能够向接收进程交付比特的速率。因为其他会话将共享沿着该网络路径的带宽，并且因为这些会话将会到达和离开，该可用吞吐量将随时间波动。这些观察导致另一种自然的服务，即运输层协议能够以某种特定的速率提供确保的可用吞吐量。使用这种服务，该应用程序能够请求 r 比特/秒的确保吞吐量，并且该运输协议能够确保可用吞吐量总是为至少 r 比特/秒。这样的确保吞吐量的服务将对许多应用程序有吸引力。例如，如果因特网电话应用程序对语音以 32kbps 的速率进行编码，那么它也必须以这个速率向网络发送数据，并向接收应用程序交付数据。如果运输协议不能提供这种吞吐量，该应用程序或以较低速率进行编码（并且接收足够的吞吐量以维持这种较低的编码速率），或它可能必须放弃发送，这是因为对于这种因特网电话应用而言，接收所需吞吐量的一半是几乎没有或根本没有用处的。具有吞吐量要求的应用程序被称为**带宽敏感的应用**（bandwidth-sensitive application）。许多当前的多媒体应用是带宽敏感的，尽管某些多媒体应用程序可能采用自适应编码技术对数字语音或视频以与当前可用带宽相匹配的速率进行编码。

带宽敏感的应用具有特定的吞吐量要求，而**弹性应用**（elastic application）能够根据情况或多或少地利用可供使用的吞吐量。电子邮件、文件传输以及 Web 传送都属于弹性应用。当然，吞吐量是越多越好。有一句格言说得好，钱越多越好，人越瘦越美，我们永远不会嫌吞吐量太多的！

3. 定时

运输层协议也能提供定时保证。如同具有吞吐量保证那样，定时保证能够以多种形式实现。一个保证的例子如：发送方注入进套接字中的每个比特到达接收方的套接字不迟于 100ms。这种服务将对交互式实时应用程序有吸引力，如因特网电话、虚拟环境、电话会议和多方游戏，所有这些服务为了有效性而要求数据交付有严格的时间限制（参见第7章，[Gauthier 1999；Ramjee 1994]）。例如，在因特网电话中，较长的时延会导致会话中出现不自然的停顿；在多方游戏和虚拟互动环境中，在做出动作并看到来自环境（如来自位于端到端连接中另一端点的玩家）的响应之间，较长的时延使得它失去真实感。对于非实时的应用，较低的时延总比较高的时延好，但对端到端的时延没有严格的约束。

4. 安全性

最后，运输协议能够为应用程序提供一种或多种安全性服务。例如，在发送主机

中，运输协议能够加密由发送进程传输的所有数据，在接收主机中，运输层协议能够在将数据交付给接收进程之前解密这些数据。这种服务将在发送和接收进程之间提供机密性，以防该数据以某种方式在这两个进程之间被观察到。运输协议还能提供除了机密性以外的其他安全性服务，包括数据完整性和端点鉴别，我们将在第 8 章中详细讨论这些主题。

2.1.4 因特网提供的运输服务

至此，我们已经考虑了计算机网络能够一般性地提供的运输服务。现在我们要更为具体地考察由因特网提供的运输服务类型。因特网（更一般的是 TCP/IP 网络）为应用程序提供两个运输层协议，即 UDP 和 TCP。当你（作为一个软件开发者）为因特网创建一个新的应用时，首先要做出的决定是，选择 UDP 还是选择 TCP。每个协议为调用它们的应用程序提供了不同的服务集合。图 2-4 显示了某些所选的应用程序的服务要求。

应用	数据丢失	带宽	时间敏感
文件传输	不能丢失	弹性	不
电子邮件	不能丢失	弹性	不
Web 文档	不能丢失	弹性（几 kbps）	不
因特网电话/视频会议	容忍丢失	音频（几 kbps ~ 1 Mbps）	是，100ms
		视频（10kbps ~ 5Mbps）	
存储音频/视频	容忍丢失	同上	是，几秒
交互式游戏	容忍丢失	几 kbps ~ 10kbps	是，100ms
即时讯息	不能丢失	弹性	是和不是

图 2-4 选择的网络应用的要求

1. TCP 服务

TCP 服务模型包括面向连接服务和可靠数据传输服务。当某个应用程序调用 TCP 作为其运输协议时，该应用程序就能获得来自 TCP 的这两种服务。

- 面向连接的服务：在应用层数据报文开始流动之前，TCP 让客户和服务端互相交换运输层控制信息。这个所谓的握手过程提示客户和服务端，使它们为大量分组的到来做好准备。在握手阶段后，一个 TCP 连接（TCP connection）就在两个进程的套接字之间建立了。这条连接是全双工的，即连接双方的进程可以在此连接上同时进行报文收发。当应用程序结束报文发送时，必须拆除该连接。在第 3 章中我们将详细讨论面向连接的服务，并分析它是如何实现的。
- 可靠的数据传送服务：通信进程能够依靠 TCP，无差错、按适当顺序交付所有发送的数据。当应用程序的一端将字节流传进套接字时，它能够依靠 TCP 将相同的字节流交付给接收方的套接字，而没有字节的丢失和冗余。

TCP 协议还具有拥塞控制机制，这种服务不一定能为通信进程带来直接好处，但能为因特网带来整体好处。当发送方和接收方之间的网络出现拥塞时，TCP 的拥塞控制机制会抑制发送进程（客户或服务端）。如我们将在第 3 章中所见，TCP 拥塞控制也试图限制每个 TCP 连接，使它们达到公平共享网络带宽的目的。

关注安全性

TCP 安全

无论 TCP 还是 UDP 都没有提供任何加密机制，这就是说发送进程传进其套接字的数据，与经网络传送到目的进程的数据相同。因此，举例来说如果某发送进程以明文方式（即没有加密）发送了一个口令进入它的套接字，该明文口令将经过发送方与接收方之间的所有链路传送，这就可能在任何中间链路被嗅探和发现。因为隐私和其他安全问题对许多应用而言已经成为至关重要的问题，所以因特网界已经研制了 TCP 的加强版本，称为安全套接字层（Secure Sockets Layer, SSL）。用 SSL 加强后的 TCP 不仅能够做传统的 TCP 所能做的一切，而且提供了关键的进程到进程的安全性服务，包括加密、数据完整性和端点鉴别。我们强调 SSL 不是与 TCP 和 UDP 在相同层次上的第三种因特网运输协议，而是一种对 TCP 的加强，这种强化是在应用层上实现的。特别是，如果一个应用程序要使用 SSL 的服务，它需要在该应用程序的客户端和服务端包括 SSL 代码（利用现有的、高度优化的库和类）。SSL 有它自己的套接字 API，这类似于传统的 TCP 套接字 API。当一个应用使用 SSL 时，发送进程向 SSL 套接字传递明文数据；在发送主机中的 SSL 则加密该数据并将加密的数据传递给 TCP 套接字。加密的数据经因特网传送到接收进程中的 TCP 套接字。该接收套接字将加密数据传递给 SSL，由其进行解密。最后，SSL 通过它的 SSL 套接字将明文数据传递给接收进程。我们将在第 8 章中更为详细地讨论 SSL。

2. UDP 服务

UDP 是一种不提供不必要服务的轻量级运输协议，它仅提供最小服务。UDP 是无连接的，因此在两个进程通信前没有握手过程。UDP 协议提供一种不可靠数据传送服务，也就是说，当进程将一个报文发送进 UDP 套接字时，UDP 协议并不保证该报文将到达接收进程。不仅如此，到达接收进程的报文也可能是乱序到达的。

UDP 没有包括拥塞控制机制，所以 UDP 的发送端可以用它选定的任何速率向其下层（网络层）注入数据。（然而，值得注意的是实际端到端吞吐量可能小于这种速率，这可能是由于中间链路的带宽受限或因为拥塞而造成的。）

3. 因特网运输协议所不提供的服务

我们已经从 4 个方面组织了运输协议服务：可靠数据传输、吞吐量、定时和安全性。TCP 和 UDP 提供了这些服务中的哪些呢？我们已经注意到 TCP 提供了可靠的端到端数据传送。并且我们也知道 TCP 在应用层可以很容易地用 SSL 来加强以提供安全服务。但在我们对 TCP 和 UDP 的简要描述中，明显地缺少了对吞吐量或定时保证的讨论，即这些服务目前的因特网运输协议并没有提供。这是否意味着诸如因特网电话这样的时间敏感应用不能运行在今天的因特网上呢？答案显然是否定的，因为在因特网上运行时间敏感应用已经有多多年了。这些应用经常工作得相当好，因为它们已经被设计成尽最大可能对付这种保证的缺乏。我们将在第 7 章中研究几种设计技巧。无论如何，在时延过大或端到端吞吐量受限时，好的设计也是有限制的。总之，今天的因特网通常能够为时间敏感应用提供满意的服务，但它不能提供任何定时或带宽保证。

图 2-5 指出了一些流行的因特网应用所使用的运输协议。可以看到，电子邮件、远程终端访问、Web、文件传输都使用了 TCP。这些应用选择 TCP 的最主要原因是 TCP 提供了可靠数据传输服务，确保所有数据最终到达目的地。因为因特网电话应用（如 Skype）通常能够容忍某些丢失但要求达到一定的最小速率才能有效工作，所以因特网电话应用的开发者通常愿意将该应用运行在 UDP 上，从而设法避开 TCP 的拥塞控制机制和分组开销。但因为许多防火墙被配置成阻挡（大多数类型的）UDP 流量，所以因特网电话应用通常设计成如果 UDP 通信失败就使用 TCP 作为备份。

应用	应用层协议	支撑的运输协议
电子邮件	SMTP [RFC 5321]	TCP
远程终端访问	Telnet [RFC 854]	TCP
Web	HTTP [RFC 2616]	TCP
文件传输	FTP [RFC 959]	TCP
流式多媒体	HTTP（如 YouTube）	TCP
因特网电话	SIP [RFC 3261]、RTP [RFC 3550] 或专用的（如 Skype）	UDP 或 TCP

图 2-5 流行的因特网应用及其应用层协议和支撑的运输协议

2.1.5 应用层协议

我们刚刚学习了通过把报文发送进套接字使网络进程间实现相互通信。但是如何构造这些报文？在这些报文中的各个字段的含义是什么？进程何时发送这些报文？这些问题将我们带进应用层协议的范围。应用层协议（application-layer protocol）定义了运行在不同端系统上的应用程序进程如何相互传递报文。特别是应用层协议定义了：

- 交换的报文类型，例如请求报文和响应报文。
- 各种报文类型的语法，如报文中的各个字段及这些字段是如何描述的。
- 字段的语义，即这些字段中包含的信息的含义。
- 一个进程何时以及如何发送报文，对报文进行响应的规则。

有些应用层协议是由 RFC 文档定义的，因此它们位于公共域中。例如，Web 的应用层协议 HTTP（超文本传输协议 [RFC 2616]）就作为一个 RFC 可供使用。如果浏览器开发者遵从 HTTP RFC 规则，所开发出的浏览器就能访问任何遵从该文档标准的 Web 服务器并获取相应 Web 页面。还有很多别的应用层协议是专用的，有意不为公共域使用。例如，Skype 使用了专用的应用层协议。

区分网络应用和应用层协议是很重要的。应用层协议只是网络应用的一部分（尽管从我们的角度看，它是应用非常重要的一部分）。我们来看一些例子。Web 是一种客户 - 服务器应用，它允许客户按照需求从 Web 服务器获得文档。该 Web 应用有很多组成部分，包括文档格式的标准（即 HTML）、Web 浏览器（如 Firefox 和 Microsoft Internet Explorer）、Web 服务器（如 Apache、Microsoft 服务器程序），以及一个应用层协议。Web 的应用层协议是 HTTP，它定义了浏览器和 Web 服务器之间传输的报文格式和序列。因此，HTTP 只是 Web 应用的一个部分（尽管是重要部分）。举另外一个例子，因特网电子邮件应用也有很多组成部分，包括能容纳用户邮箱的邮件服务器、允许用户读取和生成邮件的邮件客户程序（如 Microsoft Outlook）、定义电子邮件报文结构的标准、定义报文如何在服务器之间以及如何在服务器与邮件客户程序之间传递的应用层协议、定义如何对报文首部的内容

进行解释的应用层协议。用于电子邮件的主要应用层协议就是 SMTP（简单邮件传输协议 [RFC5321]）。因此，电子邮件的首要应用层协议 SMTP 也只是电子邮件应用的一个部分（尽管是重要部分）。

2.1.6 本书涉及的网络应用

每天都有新的公共域或者专用域因特网应用被开发出来。我们不愿像百科全书一样涉及大量的因特网应用，而是选择其中几种重要而流行的应用加以关注。在本章中我们详细讨论 5 种重要的应用：Web、文件传输、电子邮件、目录服务和 P2P。我们首先讨论 Web 应用，不仅因为它是极为流行的应用，而且因为它的应用层协议 HTTP 相对比较简单并且易于理解。讨论完 Web，我们简要地讨论 FTP，因为它与 HTTP 形成了很好的对照。我们接下来讨论电子邮件应用，这是因特网上最早的招人喜爱的应用程序。说电子邮件比 Web 更复杂，是因为它使用了多个而不是一个应用层协议。在电子邮件之后，我们讨论 DNS，它为因特网提供目录服务。大多数用户不直接与 DNS 打交道，而是通过其他的应用（包括 Web、文件传输和电子邮件）间接使用它。DNS 很好地说明了一种核心的网络功能（网络名字到网络地址的转换）是怎样在因特网的应用层实现的。本章最后我们讨论了几种 P2P 应用，重点关注文件共享应用、分布式查找服务。在第 7 章中，我们将涉及多媒体应用，包括流式视频和 IP 电话。

2.2 Web 和 HTTP

20 世纪 90 年代以前，因特网的主要使用者还是研究人员、学者和大学生，他们登录远程主机，在本地主机和远程主机之间传输文件，收发新闻，收发电子邮件。尽管这些应用非常有用（并且继续如此），但是因特网基本上不为学术界和研究界之外所知。到了 20 世纪 90 年代初期，一个主要的新型应用即万维网（World Wide Web）登上了舞台 [Berners-Lee 1994]。Web 是一个引起公众注意的因特网应用，它极大地改变了人们与工作环境内外交流的方式。它将因特网从只是很多数据网之一的地位提升为仅有的一个数据网。

也许对大多数用户来说，最具有吸引力的就是 Web 的按需操作。当用户需要时，就能得到所想要的内容。这不同于无线电广播和电视，迫使用户只能收听、收看内容提供者提供的节目。除了可以按需操作以外，Web 还有很多让人们喜欢和珍爱的特性。任何人使信息在 Web 上可用都非常简单，即只需要极低的费用就能成为出版人。超链接和搜索引擎帮助我们在 Web 站点的海洋里导航。图形化的界面刺激着我们的感官。表单、Java 小程序和很多其他的装置，使我们可以与 Web 页面和站点进行交互。并且 Web 为许多在 2003 年以后出现的招人喜爱的应用提供了平台，这些应用包括 YouTube、Gmail 和 Face book（脸谱）。

2.2.1 HTTP 概况

Web 的应用层协议是超文本传输协议（HyperText Transfer Protocol，HTTP），它是 Web 的核心，在 [RFC 1945] 和 [RFC 2616] 中进行了定义。HTTP 由两个程序实现：一个客户程序和一个服务器程序。客户程序和服务器程序运行在不同的端系统中，通过交换

HTTP 报文进行会话。HTTP 定义了这些报文的结构以及客户和服务进行报文交换的方式。在详细解释 HTTP 之前，应当回顾某些 Web 术语。

Web 页面（Web page）（也叫文档）是由对象组成的。一个对象（object）只是一个文件，诸如一个 HTML 文件、一个 JPEG 图形、一个 Java 小程序或一个视频片段这样的文件，且它们可通过一个 URL 地址寻址。多数 Web 页面含有一个 HTML 基本文件（base HTML file）以及几个引用对象。例如，如果一个 Web 页面包含 HTML 文本和 5 个 JPEG 图形，那么这个 Web 页面有 6 个对象：一个 HTML 基本文件加 5 个图形。HTML 基本文件通过对象的 URL 地址引用页面中的其他对象。每个 URL 地址由两部分组成：存放对象的服务器主机名和对象的路径名。例如，URL 地址 `http://www.someSchool.edu/someDepartment/picture.gif`，其中的 `www.someSchool.edu` 就是主机名，`/someDepartment/picture.gif` 就是路径名。因为 **Web 浏览器**（Web browser）（例如 Internet Explorer 和 Firefox）实现了 HTTP 的客户端，所以在 Web 环境中我们经常交替使用“浏览器”和“客户”这两个术语。**Web 服务器**（Web server）实现了 HTTP 的服务器端，它用于存储 Web 对象，每个对象由 URL 寻址。流行的 Web 服务器有 Apache 和 Microsoft Internet Information Server（微软互联网信息服务器）。

HTTP 定义了 Web 客户向 Web 服务器请求 Web 页面的方式，以及服务器向客户传送 Web 页面的方式。我们稍后详细讨论客户和服务器的交互过程，而其基本思想在图 2-6 中进行了图示。当用户请求一个 Web 页面（如点击一个超链接）时，浏览器向服务器发出对该页面中所包含对象的 HTTP 请求报文，服务器接收到请求并用包含这些对象的 HTTP 响应报文进行响应。

HTTP 使用 TCP 作为它的支撑运输协议（而不是在 UDP 上运行）。HTTP 客户首先发起一个与服务器的 TCP 连接。一旦连接建立，该

浏览器和服务器进程就可以通过套接字接口访问 TCP。如同在 2.1 节中描述的那样，客户端的套接字接口是客户进程与 TCP 连接之间的门，在服务器端的套接字接口则是服务器进程与 TCP 连接之间的门。客户向它的套接字接口发送 HTTP 请求报文并从它的套接字接口接收 HTTP 响应报文。类似地，服务器从它的套接字接口接收 HTTP 请求报文和向它的套接字接口发送 HTTP 响应报文。一旦客户向它的套接字接口发送了一个请求报文，该报文就脱离了客户控制并进入 TCP 的控制。2.1 节讲过，TCP 为 HTTP 提供可靠数据传输服务。这意味着，一个客户进程发出的每个 HTTP 请求报文最终能完整地到达服务器；类似地，服务器进程发出的每个 HTTP 响应报文最终能完整地到达客户。这里我们看到了分层体系结构最大的优点，即 HTTP 协议不用担心数据丢失，也不关注 TCP 从网络的数据丢失和乱序故障中恢复的细节。那是 TCP 以及协议栈较低层协议的工作。

注意到下列现象很重要：服务器向客户发送被请求的文件，而不存储任何关于该客户的状态信息。假如某个特定的客户在短短的几秒钟内两次请求同一个对象，服务器并不会因为刚刚为该客户提供了该对象就不再做出反应，而是重新发送该对象，就像服务器已经

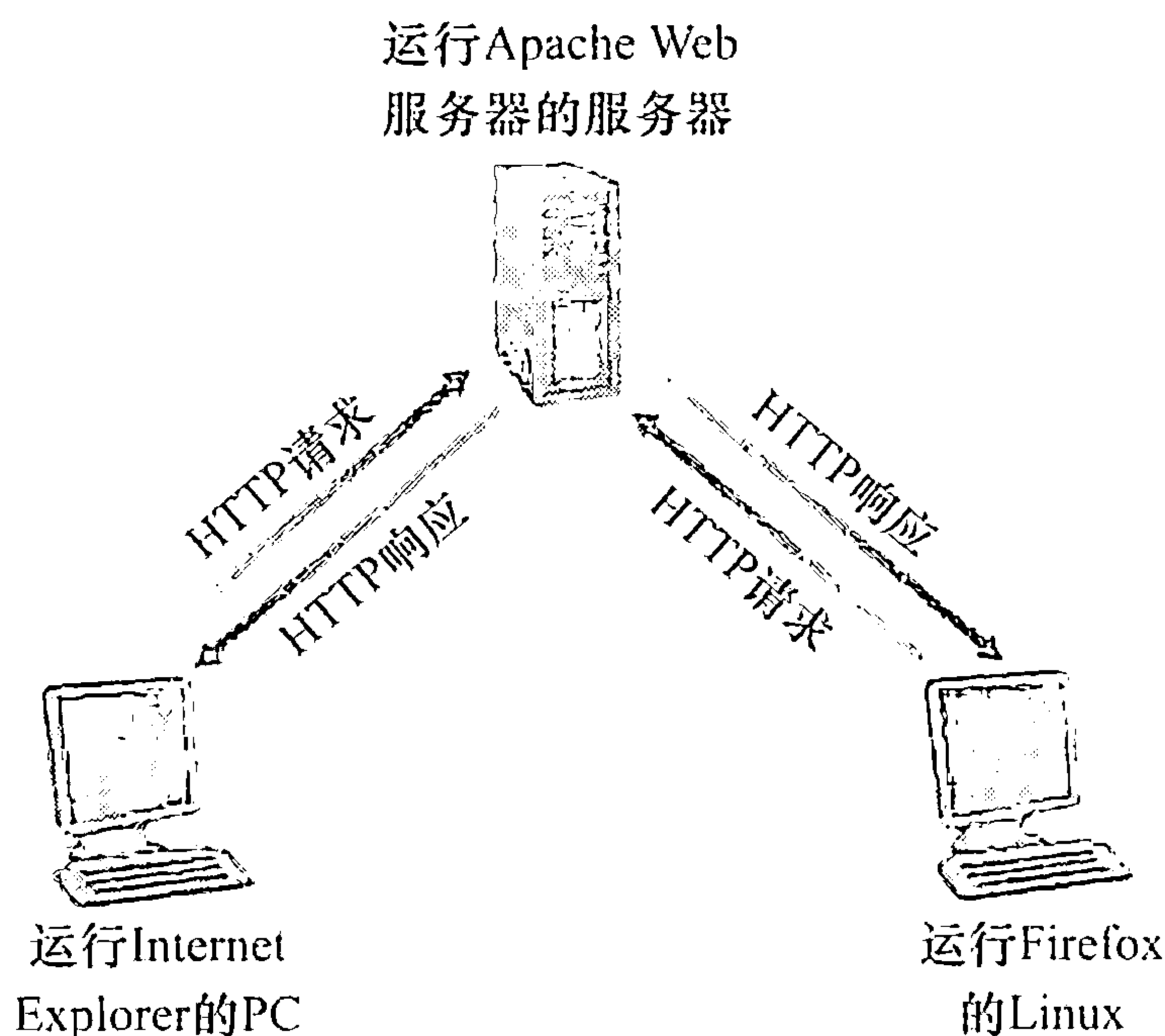


图 2-6 HTTP 的请求 - 响应行为

完全忘记不久之前所做过的事一样。因为 HTTP 服务器并不保存关于客户的任何信息，所以我们说 HTTP 是一个无状态协议（stateless protocol）。我们同时也注意到 Web 使用了客户 - 服务器应用程序体系结构（如 2.1 节所述）。Web 服务器总是打开的，具有一个固定的 IP 地址，且它服务于可能来自数以百万计的不同浏览器的请求。

2.2.2 非持续连接和持续连接

在许多因特网应用程序中，客户和服务器的通信在一个相当长的时间范围内进行，其中客户发出一系列请求并且服务器对每个请求进行响应。依据应用程序以及该应用程序的使用方式，这一系列请求可以以规则的间隔周期性地或者间断性地一个接一个发出。当这种客户 - 服务器的交互是经 TCP 进行的，应用程序的研制者就需要做一个重要决定，即每个请求/响应对是经一个单独的 TCP 连接发送，还是所有的请求及其响应经相同的 TCP 连接发送呢？采用前一种方法，该应用程序被称为使用非持续连接（non-persistent connection）；采用后一种方法，该应用程序被称为使用持续连接（persistent connection）。为了深入地理解该设计问题，我们研究在特定的应用程序即 HTTP 的情况下持续连接的优点和缺点，HTTP 既能够使用非持续连接，也能够使用持续连接。尽管 HTTP 在其默认方式下使用持续连接，HTTP 客户和服务器的配置也能配置成使用非持续连接。

1. 采用非持续连接的 HTTP

我们看看在非持续连接情况下，从服务器向客户传送一个 Web 页面的步骤。假设该页面含有一个 HTML 基本文件和 10 个 JPEG 图形，并且这 11 个对象位于同一台服务器上。该 HTML 文件的 URL 为：`http://www.someSchool.edu/someDepartment/home.index`。

我们看看发生了什么情况：

- HTTP 客户进程在端口号 80 发起一个到服务器 `www.someSchool.edu` 的 TCP 连接，该端口号是 HTTP 的默认端口。在客户和服务器的套接字与该连接相关联。
- HTTP 客户经它的套接字向该服务器发送一个 HTTP 请求报文。请求报文中包含了路径名 `/someDepartment/home.index`（后面我们会详细讨论 HTTP 报文）。
- HTTP 服务器进程经它的套接字接收该请求报文，从其存储器（RAM 或磁盘）中检索出对象 `www.someSchool.edu/someDepartment/home.index`，在一个 HTTP 响应报文中封装对象，并通过其套接字向客户发送响应报文。
- HTTP 服务器进程通知 TCP 断开该 TCP 连接。（但是直到 TCP 确认客户已经完整地收到响应报文为止，它才会实际中断连接。）
- HTTP 客户接收响应报文，TCP 连接关闭。该报文指出封装的对象是一个 HTML 文件，客户从响应报文中提取出该文件，检查该 HTML 文件，得到对 10 个 JPEG 图形的引用。
- 对每个引用的 JPEG 图形对象重复前 4 个步骤。

当浏览器收到 Web 页面后，显示给用户。两个不同的浏览器也许会以不同的方式解释（即向用户显示）该页面。HTTP 与客户如何解释一个 Web 页面毫无关系。HTTP 规范（[RFC 1945] 和 [RFC 2616]）仅定义了 HTTP 客户程序与 HTTP 服务器程序之间的通信协议。

上面的步骤举例说明了非持续连接的使用，其中每个 TCP 连接在服务器发送一个对象后关闭，即该连接并不为其他的对象而持续下来。值得注意的是每个 TCP 连接只传输一个请求报文和一个响应报文。因此在本例中，当用户请求该 Web 页面时，要产生 11 个 TCP 连接。

在上面描述的步骤中，我们有意没有明确客户获得这 10 个 JPEG 图形对象是使用 10 个串行的 TCP 连接，还是某些 JPEG 对象使用了一些并行的 TCP 连接。事实上，用户能够配置现代浏览器以控制并行度。在默认方式下，大部分浏览器打开 5 ~ 10 个并行的 TCP 连接，而每条连接处理一个请求响应事务。如果用户愿意，最大并行连接数可以设置为 1，这样 10 条连接就会串行建立。我们在下一章会看到，使用并行连接可以缩短响应时间。

在继续讨论之前，我们来简单估算一下从客户请求 HTML 基本文件起到该客户收到整个文件止所花费的时间。为此，我们给出往返时间（Round-Trip Time, RTT）的定义，该时间是指一个短分组从客户到服务器然后再返回客户所花费的时间。RTT 包括分组传播时延、分组在中间路由器和交换机上的排队时延以及分组处理时延（这些在 1.4 节已经讨论过）。现在考虑当用户点击超链接时会发生什么现象。如图 2-7 所示，这引起浏览器在它和 Web 服务器之间发起一个 TCP 连接；这涉及一次“三次握手”过程，即客户向服务器发送一个小 TCP 报文段，服务器用一个小 TCP 报文段做出确认和响应，最后，客户向服务器返回确认。三次握手中前两个部分所耗费的时间占用了一个 RTT。完成了三次握手的前两个部分后，客户结合三次握手的第三部分（确认）向该 TCP 连接发送一个 HTTP 请求报文。一旦该请求报文到达服务器，服务器就在该 TCP 连接上发送 HTML 文件。该 HTTP 请求/响应用去了另一个 RTT。因此，粗略地讲，总的响应时间就是两个 RTT 加上服务器传输 HTML 文件的时间。

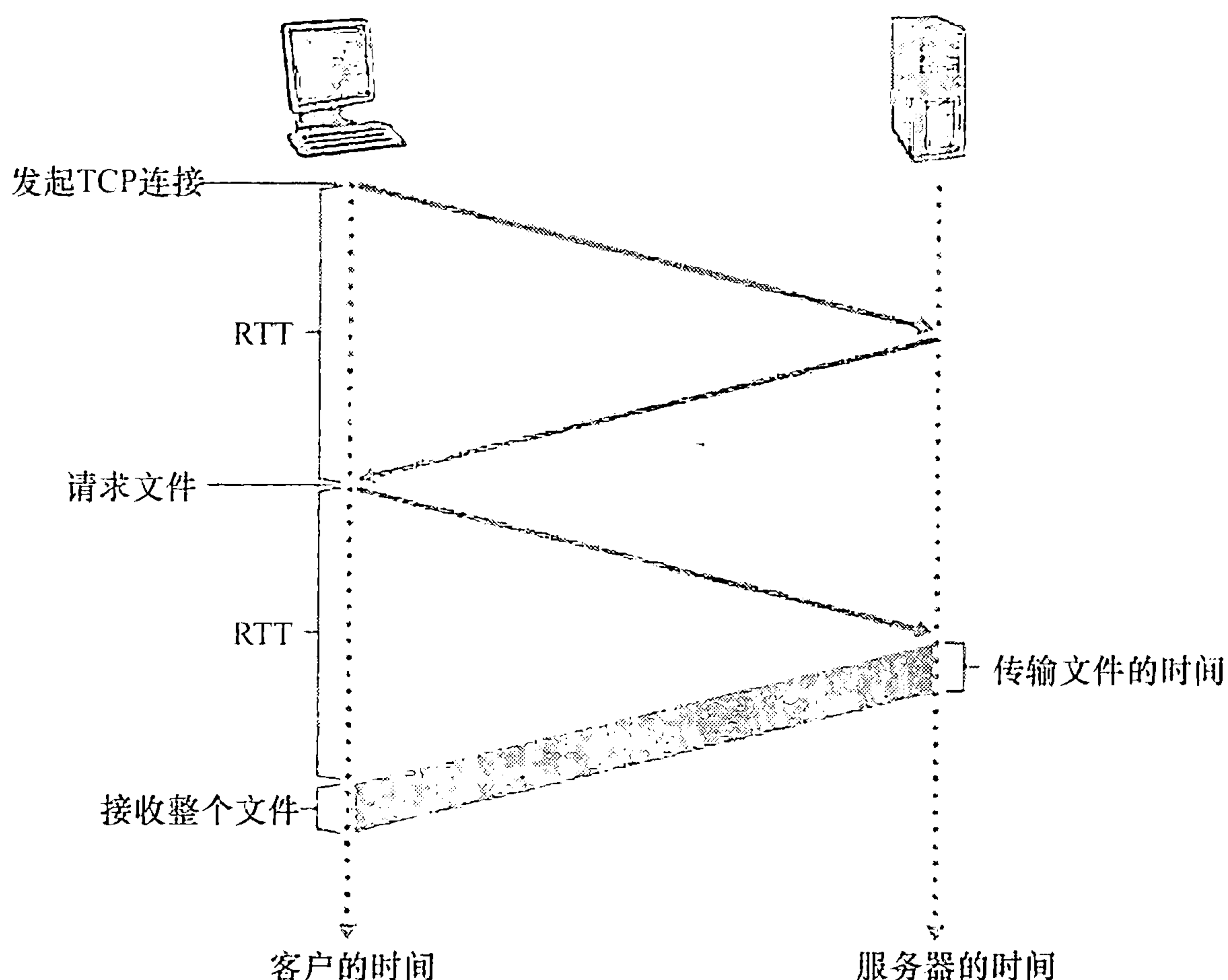


图 2-7 请求并接收一个 HTML 文件所需的时间估算

2. 采用持续连接的 HTTP

非持续连接有一些缺点。首先，必须为每一个请求的对象建立和维护一个全新的连

接。对于每个这样的连接，在客户和服务端中都要分配 TCP 的缓冲区和保持 TCP 变量，这给 Web 服务器带来了严重的负担，因为一台 Web 服务器可能同时服务于数以百计不同的客户的请求。第二，就像我们刚描述的那样，每一个对象经受两倍 RTT 的交付时延，即一个 RTT 用于创建 TCP，另一个 RTT 用于请求和接收一个对象。

在采用持续连接的情况下，服务器在发送响应后保持该 TCP 连接打开。在相同的客户与服务器之间的后续请求和响应报文能够通过相同的连接进行传送。特别是，一个完整的 Web 页面（上例中的 HTML 基本文件加上 10 个图形）可以用单个持续 TCP 连接进行传送。更有甚者，位于同一台服务器的多个 Web 页面在从该服务器发送给同一个客户时，可以在单个持续 TCP 连接上进行。可以一个接一个地发出对对象的这些请求，而不必等待对未决请求（流水线）的回答。一般来说，如果一条连接经过一定时间间隔（一个可配置的超时间隔）仍未被使用，HTTP 服务器就关闭该连接。HTTP 的默认模式是使用带流水线的持续连接。我们把量化比较持续连接和非持续连接性能的任务留作第 2、3 章的课后习题。鼓励读者阅读文献 [Heidemann 1997; Nielsen 1997]。

2.2.3 HTTP 报文格式

HTTP 规范 [RFC 1945; RFC 2616] 包含了对 HTTP 报文格式的定义。HTTP 报文有两种：请求报文和响应报文。下面讨论这两种报文。

1. HTTP 请求报文

下面提供了一个典型的 HTTP 请求报文：

```
GET /somedir/page.html HTTP/1.1
Host: www.someschool.edu
Connection: close
User-agent: Mozilla/5.0
Accept-language: fr
```

通过仔细观察这个简单的请求报文，我们就能知道很多东西。首先，我们看到该报文是用普通的 ASCII 文本书写的，这样有一定计算机知识的人都能够阅读它。其次，我们看到该报文由 5 行组成，每行由一个回车和换行符结束。最后一行后再附加一个回车换行符。虽然这个特定的报文仅有 5 行，但一个请求报文能够具有更多的行或者至少为一行。HTTP 请求报文的第一行叫做请求行（request line），其后继的行叫做首部行（header line）。请求行有 3 个字段：方法字段、URL 字段和 HTTP 版本字段。方法字段可以取几种不同的值，包括 GET、POST、HEAD、PUT 和 DELETE。绝大部分的 HTTP 请求报文使用 GET 方法。当浏览器请求一个对象时，使用 GET 方法，在 URL 字段带有请求对象的标识。在本例中，该浏览器正在请求对象/somedir/page.html。其版本字段是自解释的；在本例中，浏览器实现的是 HTTP/1.1 版本。

现在我们看看本例的首部行。首部行 Host: www.someschool.edu 指明了对象所在的主机。你也许认为该首部行是不必要的，因为在该主机中已经有一条 TCP 连接存在了。但是，如我们将在 2.2.5 节中所见，该首部行提供的信息是 Web 代理高速缓存所要求的。通过包含 Connection: close 首部行，该浏览器告诉服务器不希望麻烦地使用持续连接，它要求服务器在发送完被请求的对象后就关闭这条连接。User-agent: 首部行用来指明用户代理，即向服务器发送请求的浏览器的类型。这里浏览器类型是 Mozilla/5.0，即 Firefox 浏览器。这个首部行是有用的，因为服务器可以有效地为不同类型的用户代理

实际发送相同对象的不同版本。（每个版本都由相同的 URL 寻址。）最后，Accept-language：首部行表示用户想得到该对象的法语版本（如果服务器中有这样的对象的话）；否则，服务器应当发送它的默认版本。Accept-language：首部行仅是 HTTP 中可用的众多内容协商首部之一。

看过一个例子之后，我们再来看看如图 2-8 所示的一个请求报文的通用格式。我们看到该通用格式与我们前面的例子密切对应。然而，你可能已经注意到了在首部行（和附加的回车和换行）后有一个“实体体”（entity body）。使用 GET 方法时实体体为空，而使用 POST 方法时才使用该实体体。当用户提交表单时，HTTP 客户常常使用 POST 方法，例如当用户向搜索引擎提供搜索关键词时。使用 POST 报文时，用户仍可以向服务器请求一个 Web 页面，但 Web 页面的特定内容依赖于用户在表单字段中输入的内容。如果方法字段的值为 POST 时，则实体体中包含的就是用户在表单字段中的输入值。

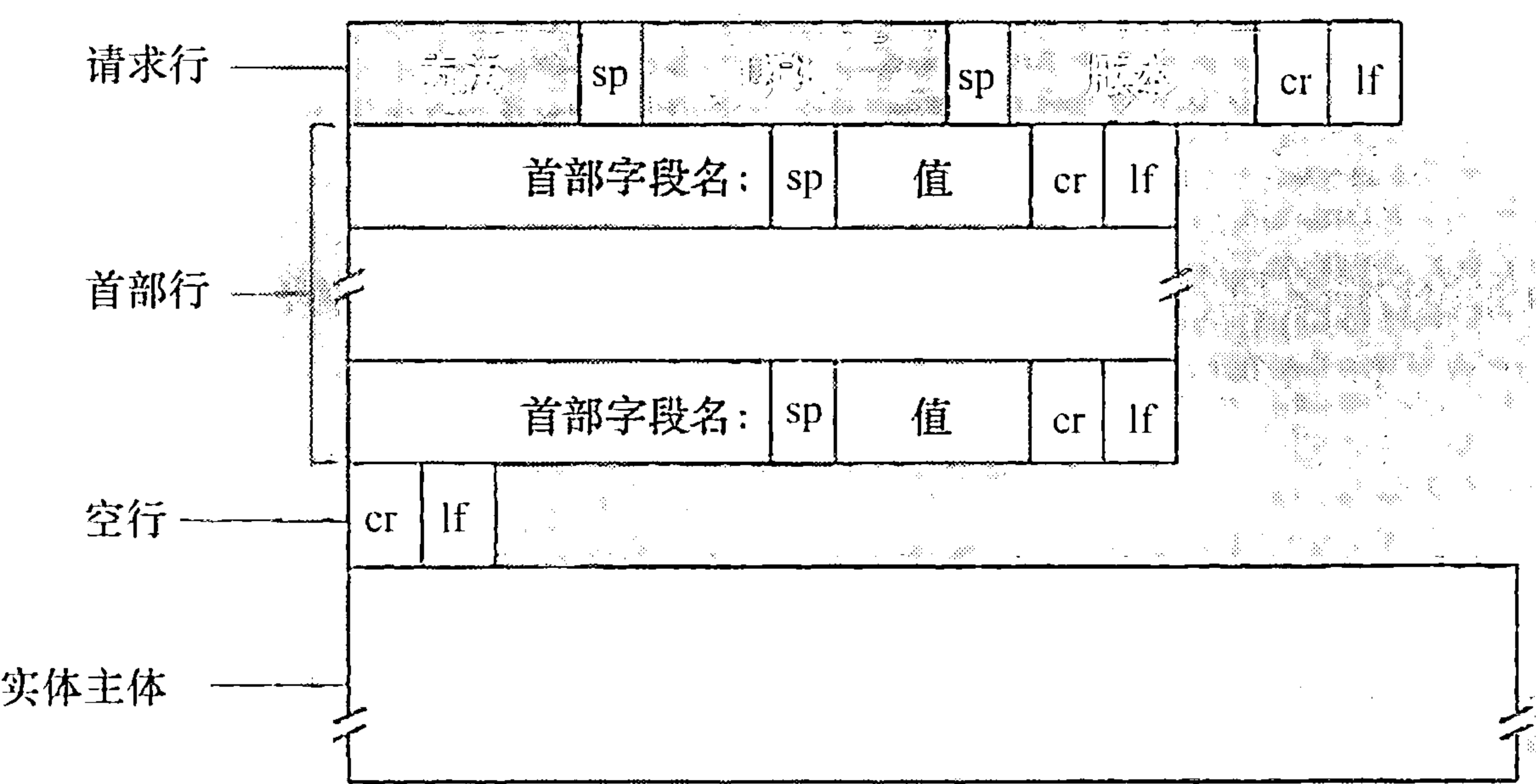


图 2-8 一个 HTTP 请求报文的通用格式

当然，如果不提“用表单生成的请求报文不是必须使用 POST 方法”这一点，那将是失职。HTML 表单经常使用 GET 方法，并在（表单字段中）所请求的 URL 中包括输入的数据。例如，一个表单使用 GET 方法，它有两个字段，分别填写的是“monkeys”和“bananas”，这样，该 URL 结构为 `www.somesite.com/animalsearch? monkeys&bananas`。在日复一日的网上冲浪中，你也许已经留意到了这种扩展的 URL。

HEAD 方法类似于 GET 方法。当服务器收到使用 HEAD 方法的请求时，将会用一个 HTTP 报文进行响应，但是并不返回请求对象。应用程序开发者常用 HEAD 方法进行调试跟踪。PUT 方法常与 Web 发行工具联合使用，它允许用户上传对象到指定的 Web 服务器上指定的路径（目录）。PUT 方法也被那些需要向 Web 服务器上传对象的应用程序使用。DELETE 方法允许用户或者应用程序删除 Web 服务器上的对象。

2. HTTP 响应报文

下面我们提供了一条典型的 HTTP 响应报文。该响应报文可以是对刚刚讨论的例子中请求报文的响应。

```
HTTP/1.1 200 OK
Connection: close
```



```
Date: Tue, 09 Aug 2011 15:44:04 GMT
Server: Apache/2.2.3 (CentOS)
Last-Modified: Tue, 09 Aug 2011 15:11:03 GMT
Content-Length: 6821
Content-Type: text/html
```

```
(data data data data data ...)
```

我们仔细看一下这个响应报文。它有三个部分：一个初始状态行（status line），6 个首部行（header line），然后是实体体（entity body）。实体体部分是报文的主要部分，即它包含了所请求的对象本身（表示为 data data data data...）。状态行有 3 个字段：协议版本字段、状态码和相应状态信息。在这个例子中，状态行指示服务器正在使用 HTTP/1.1，并且一切正常（即服务器已经找到并正在发送所请求的对象）。

我们现在来看看首部行。服务器用 Connection: close 首部行告诉客户，发送完报文后将关闭该 TCP 连接。Date: 首部行指示服务器产生并发送该响应报文的日期和时间。值得一提的是，这个时间不是指对象创建或者最后修改的时间；而是服务器从它的文件系统中检索到该对象，插入到响应报文，并发送该响应报文的时间。Server: 首部行指示该报文是由一台 Apache Web 服务器产生的，它类似于 HTTP 请求报文中的 User-agent: 首部行。Last-Modified: 首部行指示了对象创建或者最后修改的日期和时间。Last-Modified: 首部行对既可能在本地客户也可能在网络缓存服务器上的对象缓存来说非常重要。我们将很快详细地讨论缓存服务器（也叫代理服务器）。Content-Length: 首部行指示了被发送对象中的字节数。Content-Type: 首部行指示了实体体中的对象是 HTML 文本。（该对象类型应该正式地由 Content-Type: 首部行而不是用文件扩展名来指示。）

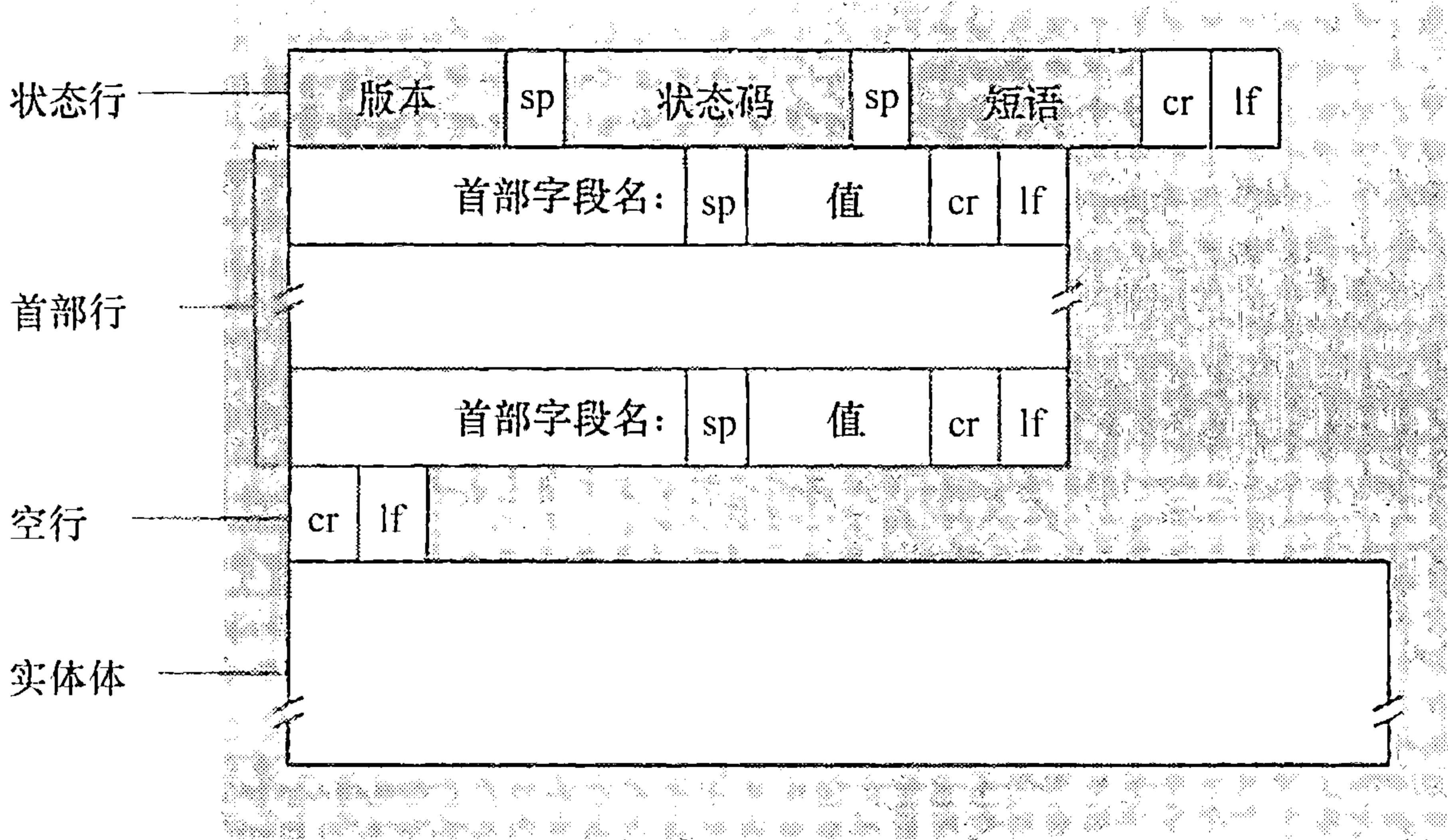


图 2-9 一个 HTTP 响应报文的通用格式

看过一个例子后，我们再来查看响应报文的通用格式（如图 2-9 所示）。该通用格式能够与前面例子中的响应报文对应起来。我们补充说明一下状态码和它们对应的短语。状态码及其相应的短语指示了请求的结果。一些常见的状态码和相关的短语包括：

- 200 OK：请求成功，信息在返回的响应报文中。
- 301 Moved Permanently：请求的对象已经被永久转移了，新的 URL 定义在响应报文的 Location：首部行中。客户软件将自动获取新的 URL。
- 400 Bad Request：一个通用差错代码，指示该请求不能被服务器理解。

- 404 Not Found: 被请求的文档不在服务器上。
- 505 HTTP Version Not Supported: 服务器不支持请求报文使用的 HTTP 协议版本。

你想看一下真实的 HTTP 响应报文吗？这正是我们高度推荐的事，而且也很容易做到。首先用 Telnet 登录到你喜欢的 Web 服务器上，接下来输入一个只有一行的请求报文去请求放在该服务器上的某些对象。例如，假设你看到命令提示，键入：

```
telnet cis.poly.edu 80
```

```
GET /~ross/ HTTP/1.1
Host: cis.poly.edu
```

（在输入最后一行后连续按两次回车。）这就打开一个到主机 cis.poly.edu 的 80 端口的 TCP 连接，并发送一个 HTTP 请求报文。你将会看到一个携带包括 Ross 教授主页的 HTML 基本文件的响应报文。如果你只是想看一下 HTTP 协议的报文行，而不是获取对象本身的话，那么可以用 HEAD 代替 GET。最后，用 /~banana/ 代替 /~ross/，看看你得到什么类型的响应报文。

在本节中，我们讨论了 HTTP 请求报文和响应报文中的一些首部行。HTTP 规范中定义了很多可以被浏览器、Web 服务器和 Web 缓存服务器插入的首部行。我们只提到了全部首部行中的少数几个，在 2.2.5 节中我们讨论网络 Web 缓存时还会涉及其他几个。一本可读性很强的文献是 [Krishnamurty 2001]，它对 HTTP 协议（包括它的首部行和状态码）进行了广泛讨论。

浏览器是如何决定在一个请求报文中包含哪些首部行的呢？Web 服务器又是如何决定在一个响应报文中包含哪些首部行呢？浏览器产生的首部行与很多因素有关，包括浏览器的类型和协议版本（例如，HTTP/1.0 浏览器将不会产生任何 1.1 版本的首部行）、浏览器的用户配置（如喜好的语言）、浏览器当前是否有一个缓存的但是可能超期的对象版本。Web 服务器的表现也类似：在产品、版本和配置上都有差异，所有这些都会影响响应报文中包含的首部行。

2.2.4 用户与服务器的交互：cookie

我们前面提到了 HTTP 服务器是无状态的。这简化了服务器的设计，并且允许工程师们去开发可以同时处理数以千计的 TCP 连接的高性能 Web 服务器。然而一个 Web 站点通常希望能够识别用户，可能是因为服务器希望限制用户的访问，或者因为它希望把内容与用户身份联系起来。为此，HTTP 使用了 cookie。cookie 在 [RFC 6265] 中定义，它允许站点对用户进行跟踪。目前大多数商务 Web 站点都使用了 cookie。

如图 2-10 所示，cookie 技术有 4 个组件：①在 HTTP 响应报文中的一个 cookie 首部行；②在 HTTP 请求报文中的一个 cookie 首部行；③在用户端系统中保留有一个 cookie 文件，并由用户的浏览器进行管理；④位于 Web 站点的一个后端数据库。使用图 2-10，我们通过一个典型的例子看看 cookie 的工作过程。假设 Susan 总是从家中 PC 使用 Internet Explorer 上网，她首次与 Amazon.com 联系。我们假定过去她已经访问过 eBay 站点。当请求报文到达该 Amazon Web 服务器时，该 Web 站点将产生一个唯一识别码，并以此作为索引在它的后端数据库中产生一个表项。接下来 Amazon Web 服务器用一个包含 Set-cookie: 首部的 HTTP 响应报文对 Susan 的浏览器进行响应，其中 Set-cookie: 首部含有该识别码。例如，该首部行可能是

```
Set-cookie: 1678
```

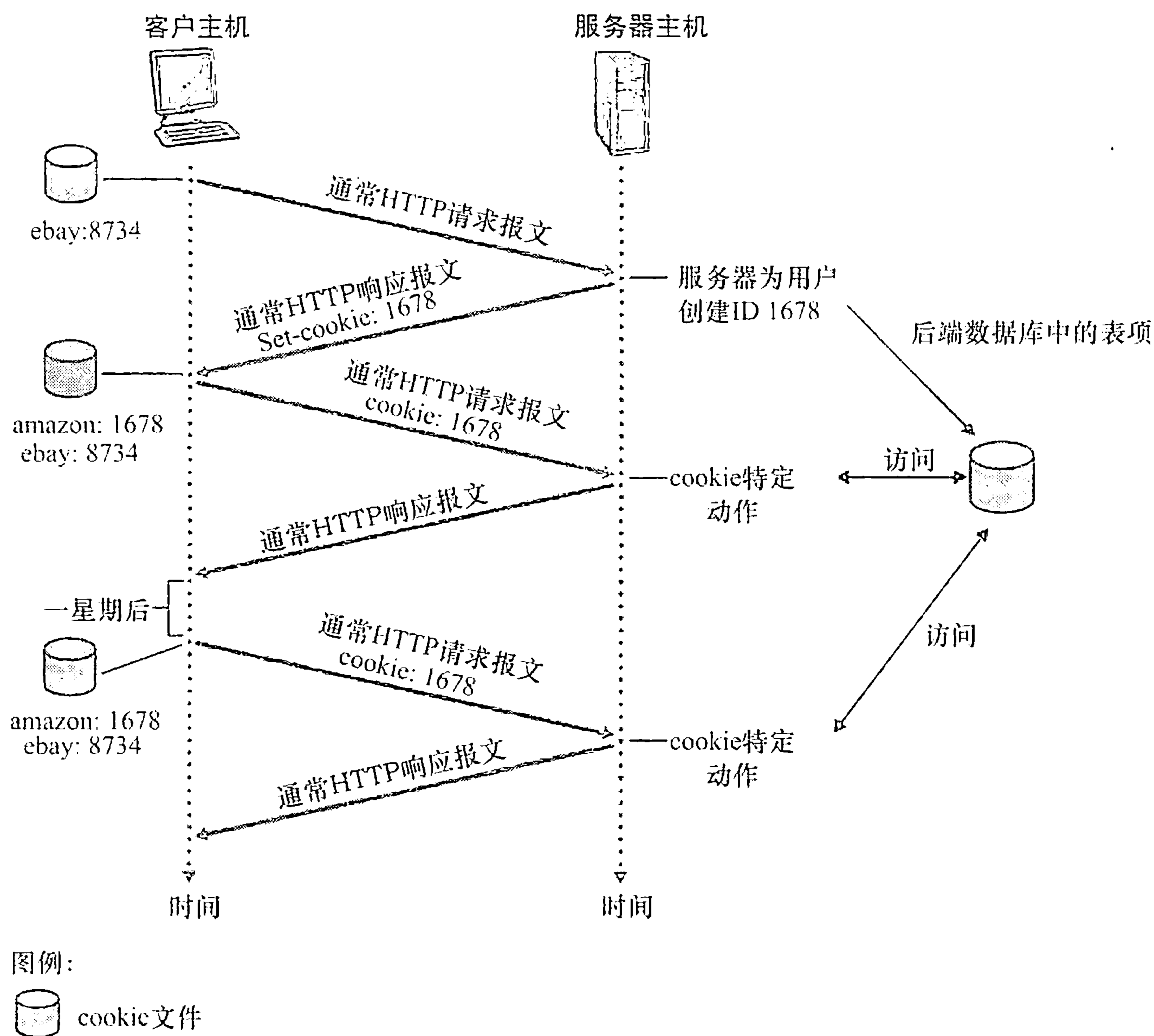



图 2-10 用 cookie 跟踪用户状态

当 Susan 的浏览器收到了该 HTTP 响应报文时，它会看到该 Set-cookie：首部。该浏览器在它管理的特定 cookie 文件中添加一行，该行包含服务器的主机名和在 Set-cookie：首部中的识别码。值得注意的是该 cookie 文件已经有了用于 eBay 的表项，因为 Susan 过去访问过该站点。当 Susan 继续浏览 Amazon 网站时，每请求一个 Web 页面，其浏览器就会从该 cookie 文件中获取她对这个网站的识别码，并放到 HTTP 请求报文中包括识别码的 cookie 首部行中。特别是，发往该 Amazon 服务器的每个 HTTP 请求报文都包括以下首部行：

Cookie: 1678

在这种方式下，Amazon 服务器可以跟踪 Susan 在 Amazon 站点的活动。尽管 Amazon Web 站点不必知道 Susan 的名字，但它确切地知道用户 1678 按照什么顺序、在什么时间、访问了哪些页面！Amazon 使用 cookie 来提供它的购物车服务，即 Amazon 能够维护 Susan 希望购买的物品列表，这样在 Susan 结束会话时可以一起为它们付费。

如果 Susan 再次访问 Amazon 站点，比如说一个星期后，她的浏览器会在其请求报文中继续放入首部行 cookie: 1678。Amazon 将根据 Susan 过去在 Amazon 访问的网页向她推荐产品。如果 Susan 也在 Amazon 注册过，即提供了她的全名、电子邮件地址、邮政地址和信用卡账号，则 Amazon 能在其数据库中包括这些信息，将她的全名与识别码相关联（以及她在过去访问过的所有页面）。这就解释了 Amazon 和其他一些电子商务网站实现“点击购物”（one-click shopping）的道理，即当 Susan 在后继的访问中选择购买某个物品

时，她不必重新输入姓名、信用卡账号或者地址等信息了。

从上述讨论中我们看到，cookie 可以用于标识一个用户。用户首次访问一个站点时，可能需要提供一个用户标识（可能是名字）。在后继会话中，浏览器向服务器传递一个 cookie 首部，从而向该服务器标识了用户。因此 cookie 可以在无状态的 HTTP 之上建立一个用户会话层。例如，当用户向一个基于 Web 的电子邮件系统（如 Hotmail）注册时，浏览器向服务器发送 cookie 信息，允许该服务器在用户与应用程序会话的过程中标识该用户。

尽管 cookie 常常能简化用户的因特网购物活动，但是它的使用仍具有争议，因为它们被认为是对用户隐私的一种侵害。如我们刚才所见，结合 cookie 和用户提供的账户信息，Web 站点可以知道许多有关用户的信息，并可能将这些信息卖给第三方。Cookie Central [Cookie Central 2012] 包括了对 cookie 争论的广泛信息。

2.2.5 Web 缓存

Web 缓存器（Web cache）也叫代理服务器（proxy server），它是能够代表初始 Web 服务器来满足 HTTP 请求的网络实体。Web 缓存器有自己的磁盘存储空间，并在存储空间中保存最近请求过的对象的副本。如图 2-11 所示，可以配置用户的浏览器，使得用户的所有 HTTP 请求首先指向 Web 缓存器。一旦某浏览器被配置，每个对某对象的浏览器请求首先被定向到该 Web 缓存器。举例来说，假设浏览器正在请求对象 `http://www.someschool.edu/campus.gif`，将会发生如下情况：

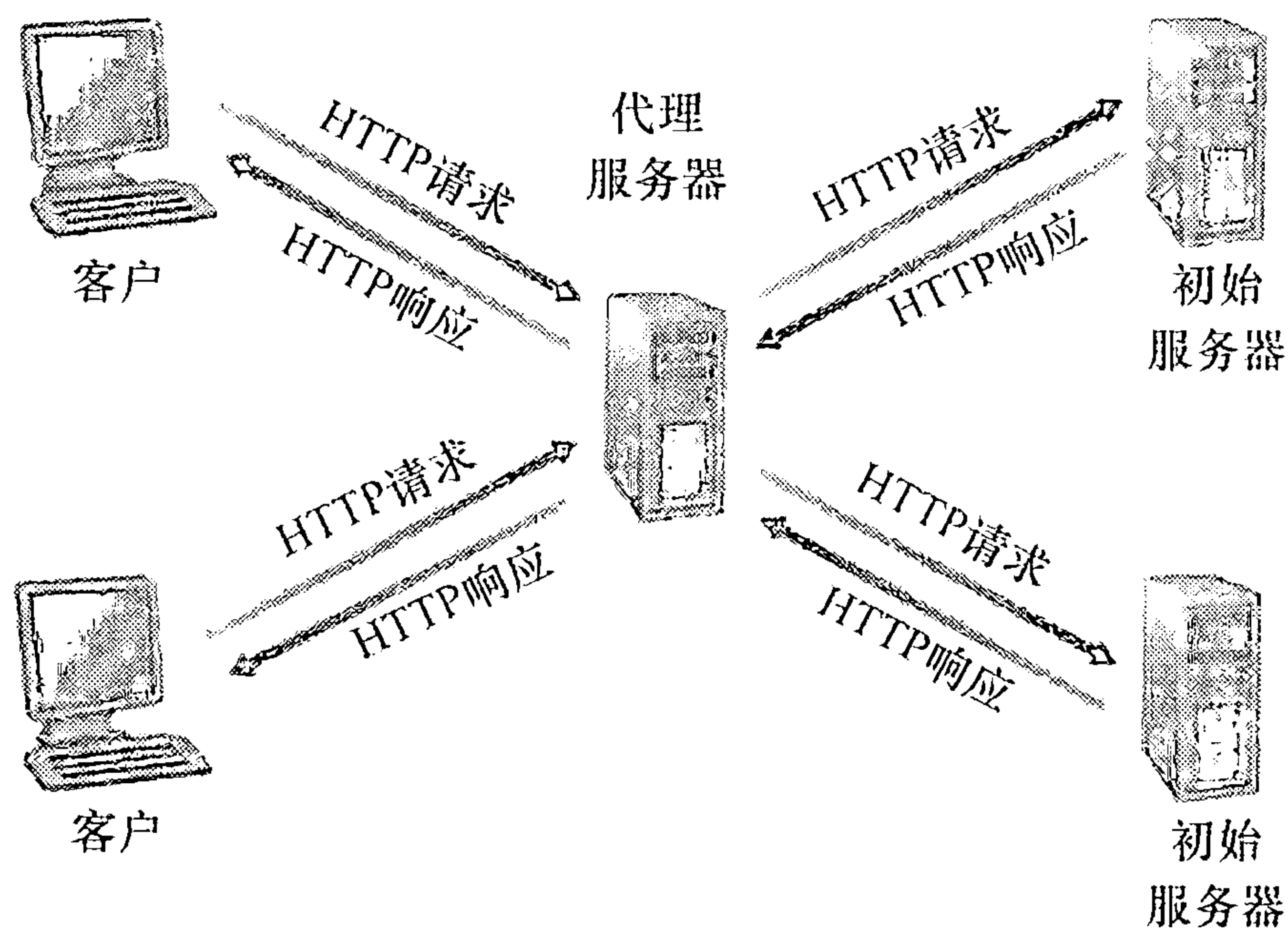


图 2-11 客户通过 Web 缓存器请求对象

- 浏览器建立一个到 Web 缓存器的 TCP 连接，并向 Web 缓存器中的对象发送一个 HTTP 请求。
- Web 缓存器进行检查，看看本地是否存储了该对象副本。如果有，Web 缓存器就向客户浏览器用 HTTP 响应报文返回该对象。
- 如果 Web 缓存器中没有该对象，它就打开一个与该对象的初始服务器（如 `www.someschool.edu`）的 TCP 连接。Web 缓存器则在这个缓存器到服务器的 TCP 连接上发送一个对该对象的 HTTP 请求。在收到该请求后，初始服务器向该 Web 缓存器发送具有该对象的 HTTP 响应。
- 当 Web 缓存器接收到该对象时，它在本地存储空间存储一份副本，并向客户的浏览器用 HTTP 响应报文发送该副本（通过现有的客户浏览器和 Web 缓存器之间的 TCP 连接）。

值得注意的是 Web 缓存器是服务器同时又是客户。当它接收浏览器的请求并发回响应时，它是一个服务器。当它向初始服务器发出请求并接收响应时，它是一个客户。

Web 缓存器通常由 ISP 购买并安装。例如，一所大学可能在它的校园网上安装一台缓

存器，并且将所有校园网上的用户浏览器配置为指向它。或者，一个主要的住宅 ISP（例如 AOL）可能在它的网络上安装一台或多台 Web 缓存器，并且预先配置其配套的浏览器指向这些缓存器。

在因特网上部署 Web 缓存器有两个原因。首先，Web 缓存器可以大大减少对客户请求的响应时间，特别是当客户与初始服务器之间的瓶颈带宽远低于客户与 Web 缓存器之间的瓶颈带宽时更是如此。如果在客户与 Web 缓存器之间有一个高速连接（情况常常如此），并且如果用户所请求的对象在 Web 缓存器上，则 Web 缓存器可以迅速将该对象交付给用户。其次，如我们马上用例子说明的那样，Web 缓存器能够大大减少一个机构的接入链路到因特网的通信量。通过减少通信量，该机构（如一家公司或者一所大学）就不必急于增加带宽，因此降低了费用。此外，Web 缓存器能从整体上大大减低因特网上的 Web 流量，从而改善了所有应用的性能。

为了深入理解缓存器带来的好处，我们考虑在图 2-12 场景下的一个例子。该图显示了两个网络，即机构（内部）网络和公共因特网的一部分。机构网络是一个高速的局域网，它的一台路由器与因特网上的一台路由器通过一条 15Mbps 的链路连接。这些初始服务器与因特网相连但位于全世界各地。假设对象的平均长度为 1Mb，从机构内的浏览器对这些初始服务器的平均访问速率为每秒 15 个请求。假设 HTTP 请求报文小到可以忽略，因而不会在网络中以及接入链路（从机构内部路由器到因特网路由器）上产生什么通信量。我们还假设在图 2-12 中从因特网接入链路一侧的路由器转发 HTTP 请求报文（在一个 IP 数据报中）开始，到它收到其响应报文（通常在多个 IP 数据报中）为止的时间平均为 2 秒。我们非正式地将该持续时延称为“因特网时延”。

总的响应时间，即从浏览器请求一个对象到接收到该对象为止的时间，是局域网时延、接入时延（即两台路由器之间的时延）和因特网时延之和。我们来粗略地估算一下这个时延。局域网上的流量强度（参见 1.4.2 节）为

$$(15 \text{ 个请求/s}) \times (1\text{Mb/请求}) / (100\text{Mbps}) = 0.15$$

然而接入链路上的流量强度（从因特网路由器到机构路由器）为

$$(15 \text{ 个请求/s}) \times (1\text{Mb/请求}) / (15\text{Mbps}) = 1$$

局域网上强度为 0.15 的通信量最多导致数十毫秒的时延，因此我们可以忽略局域网时延。然而，如在 1.4.2 节讨论的那样，如果流量强度接近 1（就像在图 2-12 中接入链路的情况那样），链路上的时延会变得非常大并且无限增长。因此，满足请求的平均响应时间将在分钟的量级上。显然，必须想办法来改进时间响应特性。

一个可能的解决办法就是增加接入链路的速率，如从 15Mbps 增加到 100Mbps。这可以将接入链路上的流量强度减少到 0.15，这样一来，两台路由器之间的链路时延也可以忽

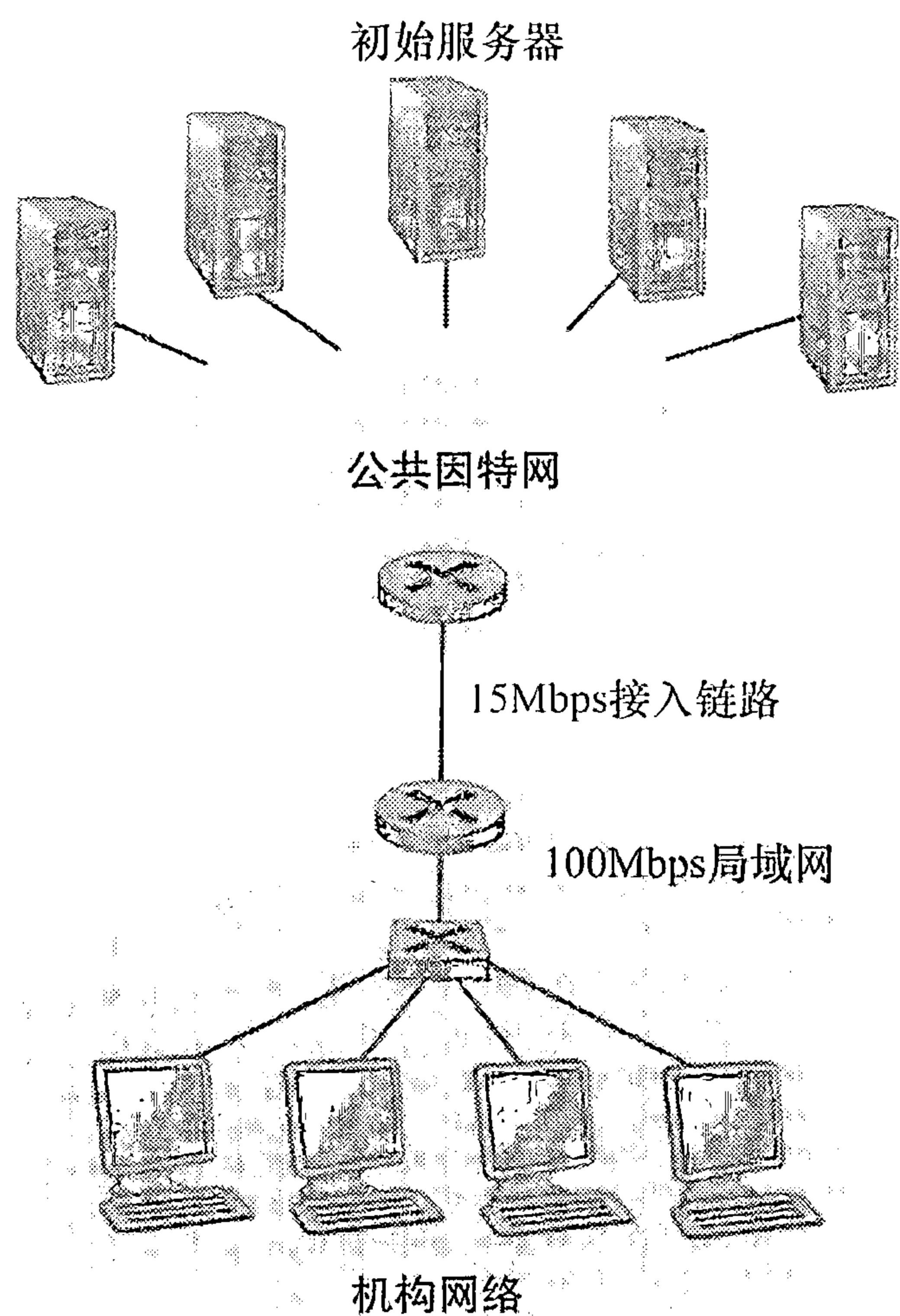


图 2-12 一个机构网络与因特网之间的瓶颈

略了。这时，总的响应时间将大约为2秒钟，即为因特网时延。但这种解决方案也意味着该机构必须将它的接入链路由15Mbps升级为100Mbps，这是一种代价很高的方案。

现在来考虑另一种解决方案，即不升级链路带宽而是在机构网络中安装一个Web缓存器。这种解决方案如图2-13所示。实践中的命中率（即由一个缓存器所满足的请求的比率）通常在0.2~0.7之间。为了便于阐述，我们假设该机构的缓存命中率为0.4。因为客户和缓存连接在一个相同的高速局域网上，这样40%的请求将几乎立即会由缓存器得到响应，时延约在10ms以内。然而，剩下的60%的请求仍然要由初始服务器来满足。但是只有60%的被请求对象通过接入链路，在接入链路上的流量强度从1.0减小到0.6。一般而言，在15Mbps链路上，当流量强度小于0.8时对应的时延较小，约为几十毫秒。这个时延与2秒因特网时延相比是微不足道的。考虑这些之后，平均时延因此为

$$0.4 \times (0.010 \text{ 秒}) + 0.6 \times (2.01 \text{ 秒})$$

这略大于1.2秒。因此，第二种解决方案提供的响应时延甚至比第一种解决方案更低，也不需要该机构升级它到因特网的链路。该机构理所当然地要购买和安装Web缓存器。除此之外其成本较低，很多缓存器使用了运行在廉价PC上的公共域软件。

通过使用内容分发网络（Content Distribution Network, CDN），Web缓存器正在因特网中发挥着越来越重要的作用。CDN公司在因特网上安装了许多地理上分散的缓存器，因而使大量流量实现了本地化。有多个共享的CDN（例如Akamai和Lime-light）和专用的CDN（例如谷歌和微软）。我们将在第7章中更为详细地讨论CDN。

2.2.6 条件GET方法

尽管高速缓存能减少用户感受到的响应时间，但也引入了一个新的问题，即存放在缓存器中的对象副本可能是陈旧的。

换句话说，保存在服务器中的对象自该副本缓存在客户上以后可能已经被修改了。幸运的是，HTTP协议有一种机制，允许缓存器证实它的对象是最新的。这种机制就是条件GET（conditional GET）方法。如果：①请求报文使用GET方法；并且②请求报文中包含一个“If-Modified-Since:”首部行。那么，这个HTTP请求报文就是一个条件GET请求报文。

为了说明GET方法的操作方式，我们看一个例子。首先，一个代理缓存器（proxy cache）代表一个请求浏览器，向某Web服务器发送一个请求报文：

```
GET /fruit/kiwi.gif HTTP/1.1
Host: www.exotiquecuisine.com
```

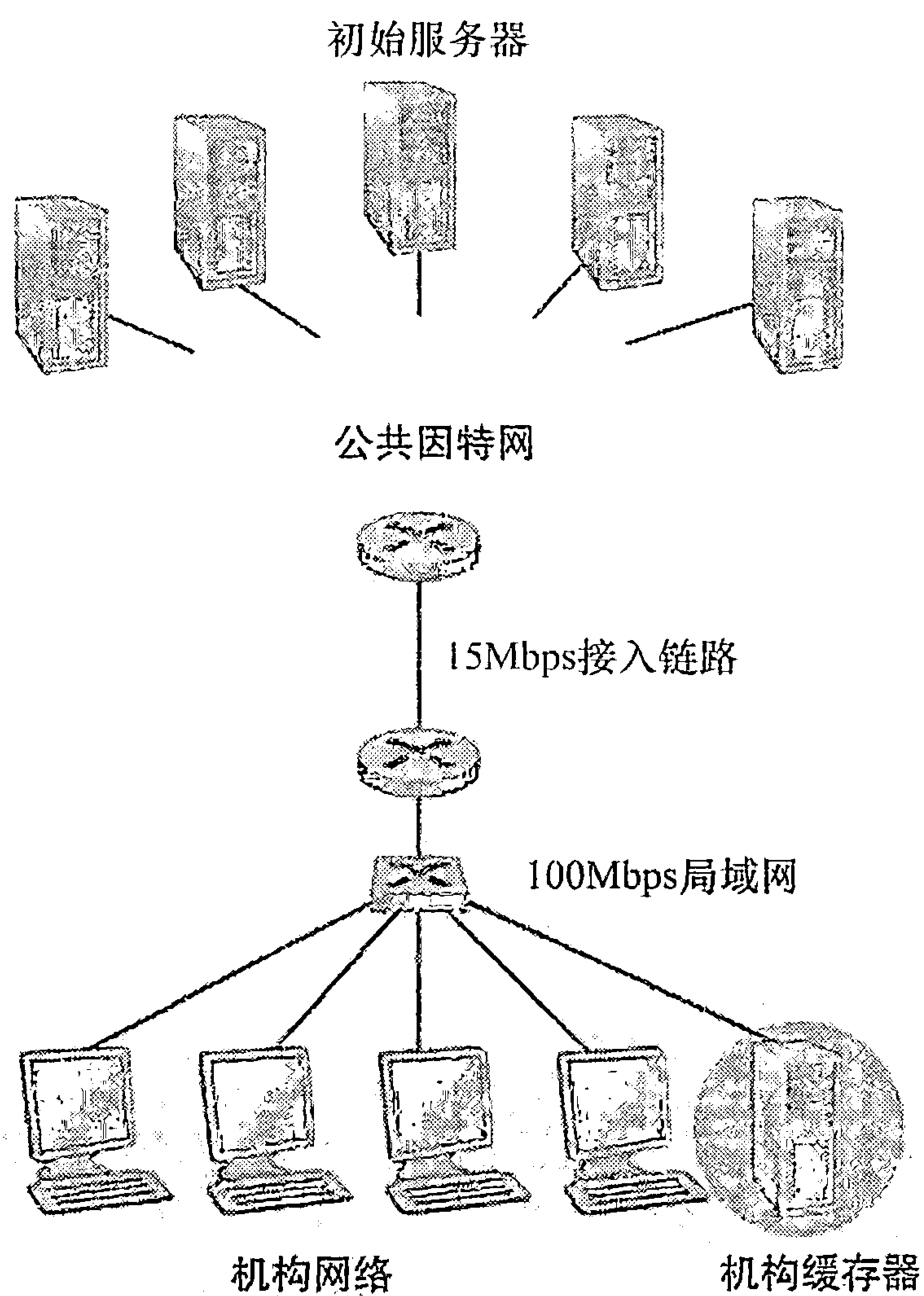


图2-13 为机构网络添加一台缓存器

其次，该 Web 服务器向缓存器发送具有被请求的对象的响应报文：

```
HTTP/1.1 200 OK
Date: Sat, 8 Oct 2011 15:39:29
Server: Apache/1.3.0 (Unix)
Last-Modified: Wed, 7 Sep 2011 09:23:24
Content-Type: image/gif
```

(data data data data data ...)

该缓存器在将对象转发到请求的浏览器的同时，也在本地缓存了该对象。重要的是，缓存器在存储该对象时也存储了最后修改日期。第三，一个星期后，另一个用户经过该缓存器请求同一个对象，该对象仍在这个缓存器中。由于在过去的一个星期中位于 Web 服务器上的该对象可能已经被修改了，该缓存器通过发送一个条件 GET 执行最新检查。具体说来，该缓存器发送：

```
GET /fruit/kiwi.gif HTTP/1.1
Host: www.exotiquecuisine.com
If-Modified-Since: Wed, 7 Sep 2011 09:23:24
```

值得注意的是 If-Modified-Since：首部行的值正好等于一星期前服务器发送的响应报文中的 Last-Modified：首部行的值。该条件 GET 报文告诉服务器，仅当自指定日期之后该对象被修改过，才发送该对象。假设该对象自 2011 年 9 月 7 日 09:23:24 后没有被修改。接下来的第四步，Web 服务器向该缓存器发送一个响应报文：

```
HTTP/1.1 304 Not Modified
Date: Sat, 15 Oct 2011 15:39:29
Server: Apache/1.3.0 (Unix)
```

(empty entity body)

我们看到，作为对该条件 GET 方法的响应，该 Web 服务器仍发送一个响应报文，但并没有在该响应报文中包含所请求的对象。包含该对象只会浪费带宽，并增加用户感受到的响应时间，特别是如果该对象很大的时候更是如此。值得注意的是在最后的响应报文中，状态行中为 304 Not Modified，它告诉缓存器可以使用该对象，能向请求的浏览器转发它（该代理缓存器）缓存的该对象副本。

我们现在完成了对 HTTP 的讨论，这是我们详细学习的第一个因特网协议（应用层协议）。我们已经学习了 HTTP 报文的格式，学习了当发送和接收这些报文时 Web 客户和服务端所采取的动作。我们还学习了一点 Web 应用程序基础设施，包括缓存、cookie 和后端数据库，所有这些都以某种方式与 HTTP 协议有关。

2.3 文件传输协议：FTP

在一个典型的 FTP 会话中，用户坐在一台主机（本地主机）前面，向一台远程主机传输（或接收来自远程主机的）文件。为使用户能访问它的远程账户，用户必须提供一个用户标识和口令。在提供了这种授权信息后，用户就能从本地文件系统向远程主机文件系统传送文件，反之亦然。如图 2-14 所示，用户通过一个 FTP 用户代理与 FTP 交互。该用户首先提供远程主机的主机名，使本地主机的 FTP 客户进程建立一个到远程主机 FTP 服务器进程的 TCP 连接。该用户接着提供用户标识和口令，作为 FTP 命令的一部分在该 TCP 连接上传送。一旦该服务器向该用户授权，用户可以将存放在本地文件系统中的某一个或者多个文件复制到远程文件系统（反之亦然）。

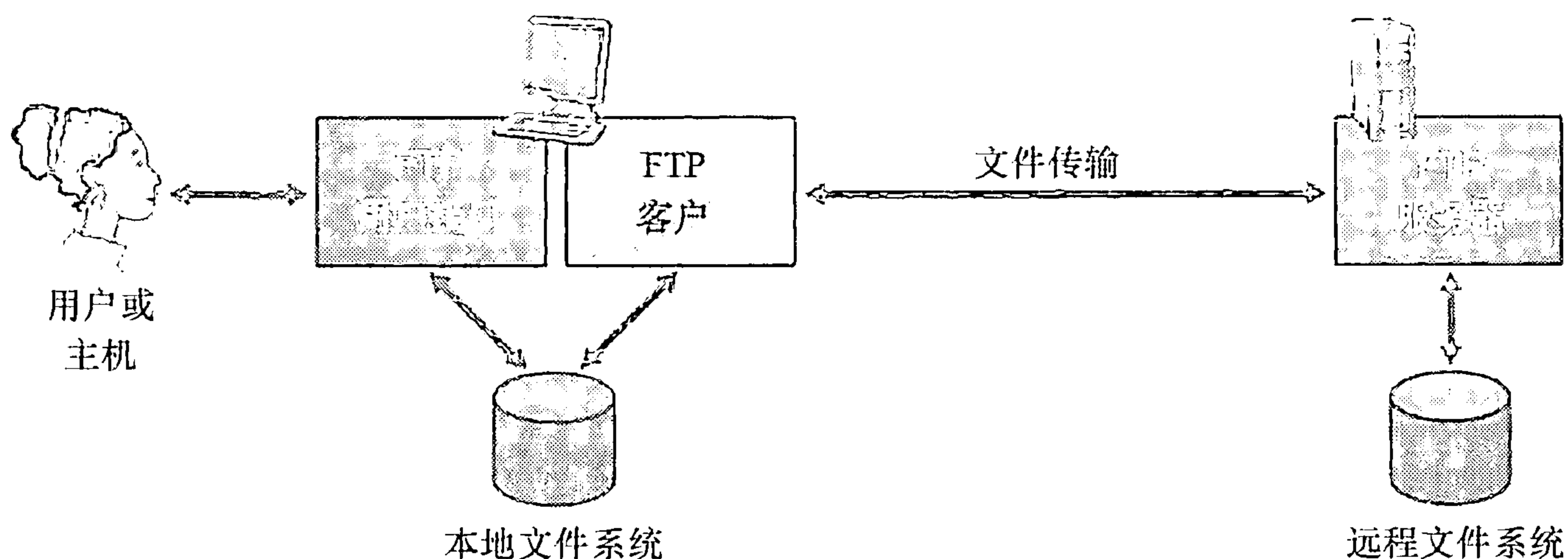


图 2-14 使用 FTP 协议在本地和远程文件系统之间移动文件

HTTP 和 FTP 都是文件传输协议，并且有很多共同的特点，例如，它们都运行在 TCP 上。然而，这两个应用层协议也有一些重要的区别。其中最显著的就是 FTP 使用了两个并行的 TCP 连接来传输文件，一个是控制连接（control connection），一个是数据连接（data connection）。控制连接用于在两主机之间传输控制信息，如用户标识、口令、改变远程目录的命令以及“存放（put）”和“获取（get）”文件的命令。数据连接用于实际发送一个文件。因为 FTP 协议使用一个独立的控制连接，所以我们也称 FTP 的控制信息是带外（out-of-band）传送的。如你所知，HTTP 协议是在传输文件的同一个 TCP 连接中发送请求和响应首部行的。因此，HTTP 也可以说是带内（in-band）发送控制信息的。FTP 协议的控制连接和数据连接如图 2-15 所示。

当用户主机与远程主机开始一个 FTP 会话时，FTP 的客户（用户）端首先在服务器 21 号端口与服务器（远程主机）端发起一个用于控制的 TCP 连接。FTP 的客户端也通过该控制连接发送用户的标识和口令，发送改变远程目录的命令。当 FTP 的服务器端从该连接上收到一个文件传输的命令后（无论是向还是来自远程主机），就发起一个到客户端的 TCP 数据连接。FTP 在该数据连接上准确地传送一个文件，然后关闭该连接。在同一个会话期间，如果用户还需要传输另一个文件，FTP 则打开另一个数据连接。因而对 FTP 传输而言，控制连接贯穿了整个用户会话期间，但是对会话中的每一次文件传输都需要建立一个新的数据连接（即数据连接是非持续的）。

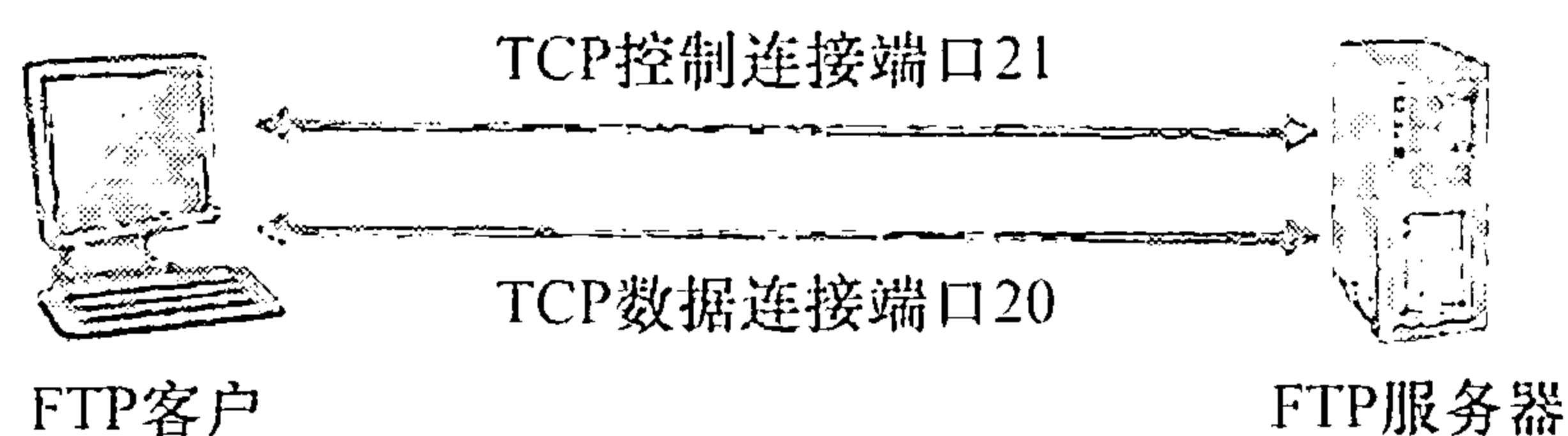


图 2-15 控制连接和数据连接

FTP 服务器必须在整个会话期间保留用户的状态（state）。特别是，服务器必须把特定的用户账户与控制连接联系起来，随着用户在远程目录树上徘徊，服务器必须追踪用户在远程目录树上的当前位置。对每个进行中的用户会话的状态信息进行追踪，大大限制了 FTP 同时维持的会话总数。而另一方面，前面讲过 HTTP 是无状态的，即它不必对任何用户状态进行追踪。

FTP 服务器必须在整个会话期间保留用户的状态（state）。特别是，服务器必须把特定的用户账户与控制连接联系起来，随着用户在远程目录树上徘徊，服务器必须追踪用户在远程目录树上的当前位置。对每个进行中的用户会话的状态信息进行追踪，大大限制了 FTP 同时维持的会话总数。而另一方面，前面讲过 HTTP 是无状态的，即它不必对任何用户状态进行追踪。

FTP 命令和回答

我们通过简要地讨论几个常见的 FTP 命令和回答来结束本节。从客户到服务器的命令和从服务器到客户的回答，都是以 7 比特 ASCII 格式在控制连接上传送的。因此，与 HTTP 协议的命令类似，FTP 协议的命令也是人可读的。为了区分连续的命令，每个命令后跟回车换行符。每个命令由 4 个大写字母 ASCII 字符组成，有些还具有可选参数。一些

较为常见的命令如下：

- USER username：用于向服务器传送用户标识。
- PASS password：用于向服务器发送用户口令。
- LIST：用于请求服务器回送当前远程目录中的所有文件列表。该文件列表是经一个（新建且非持续连接）数据连接传送的，而不是在控制 TCP 连接上传送。
- RETR filename：用于从远程主机当前目录检索（即 get）文件。该命令引起远程主机发起一个数据连接，并经该数据连接发送所请求的文件。
- STOR filename：用于在远程主机的当前目录上存放（即 put）文件。

贯穿控制连接，在用户发出的命令和 FTP 发送的命令之间通常有一一对应关系。每个命令都对应着一个从服务器发向客户的回答。回答是一个 3 位的数字，后跟一个可选信息。这与 HTTP 响应报文状态行的状态码和状态信息的结构相同。一些典型的回答连同它们可能的报文如下所示：

- 331 Username OK, Password required（用户名 OK，需要口令）。
- 125 Data connection already open; transfer starting（数据连接已经打开，开始传送）。
- 425 Can't open data connection（无法打开数据连接）。
- 452 Error writing file（写文件差错）。

有兴趣学习其他 FTP 命令和回答的读者请阅读 RFC 959。

2.4 因特网中的电子邮件

自从有了因特网，电子邮件就在因特网上流行起来。当因特网还在襁褓中时，电子邮件已经成为最为流行的应用程序 [Segaller 1998]，年复一年，它变得越来越精细，越来越强大。它是当今因特网上最重要和实用的应用程序之一。

与普通邮件一样，电子邮件是一种异步通信媒介，即当人们方便时就可以收发邮件，不必与他人的计划进行协调。与普通邮件相比，电子邮件更为快速并且易于分发，而且价格便宜。现代电子邮件具有许多强大的特性，包括具有附件、超链接、HTML 格式文本和图片的报文。

历史事件

Web 电子邮件

1995 年 12 月，在 Web “发明”后仅过了几年，Sabeer Bhatia 和 Jack Smith 拜访了因特网风险投资人 Draper Fisher Jurvetson，提出了研发一个免费的基于 Web 的电子邮件系统的建议。其基本想法是为任何想要的人分配一个免费的电子邮件账户，并且使得这个账户可以在 Web 上使用。通过用公司的 15% 份额作为交换，Draper Fisher Jurvetson 向 Bhatia 和 Smith 提供了资金，后者组建了一家公司，叫做 Hotmail。3 个全职员工和 14 个兼职人员为了自己拥有的股份而工作，1996 年 7 月，他们研发并提供了该服务。之后的一个月，他们就拥有了 100 000 名用户。1997 年 12 月，在启动该服务不到 18 个月内，Hotmail 就拥有超过 1200 万个用户，并且以 4 亿美元的价格被微软公司收购。Hotmail 公司的成功常被归结于它的“先行者优势（first-mover advantage）”和它固有的电子邮件

“病毒行销 (viral marketing)” 策略。(也许正在阅读本书的某些学生将会成为这种新人之一——构思并开发具有“先行者优势”和“病毒行销”策略特征的因特网服务。)

Web 电子邮件继续兴盛，每年都变得更为复杂和功能强大。当今最为流行的服务之一是谷歌的 Gmail，它提供了千兆字节的免费存储、先进的垃圾邮件过滤和病毒检测、电子邮件加密 (使用 SSL)、对第三方电子邮件服务的邮件接纳和面向搜索的界面。社交网络如脸谱中的异步信息转发在近年来也已经变得流行。

在本节中，我们将讨论位于因特网电子邮件的核心地位的应用层协议。在深入讨论这些应用层协议之前，我们先总体上看一看因特网电子邮件系统和它的关键组件。

图 2-16 给出了因特网电子邮件系统的总体情况。从该图中我们可以看到它有 3 个主要组成部分：用户代理 (user agent)、邮件服务器 (mail server) 和简单邮件传输协议 (Simple Mail Transfer Protocol, SMTP)。下面我们结合发送方 Alice 发电子邮件给接收方 Bob 的例子，对每个组成部分进行描述。用户代理允许用户阅读、回复、转发、保存和撰写报文。微软的 Outlook 和 Apple Mail 是电子邮件用户代理的例子。当 Alice 完成邮件撰写时，她的邮件代理向其邮件服务器发送邮件，此时邮件放在邮件服务器的外出报文队列中。

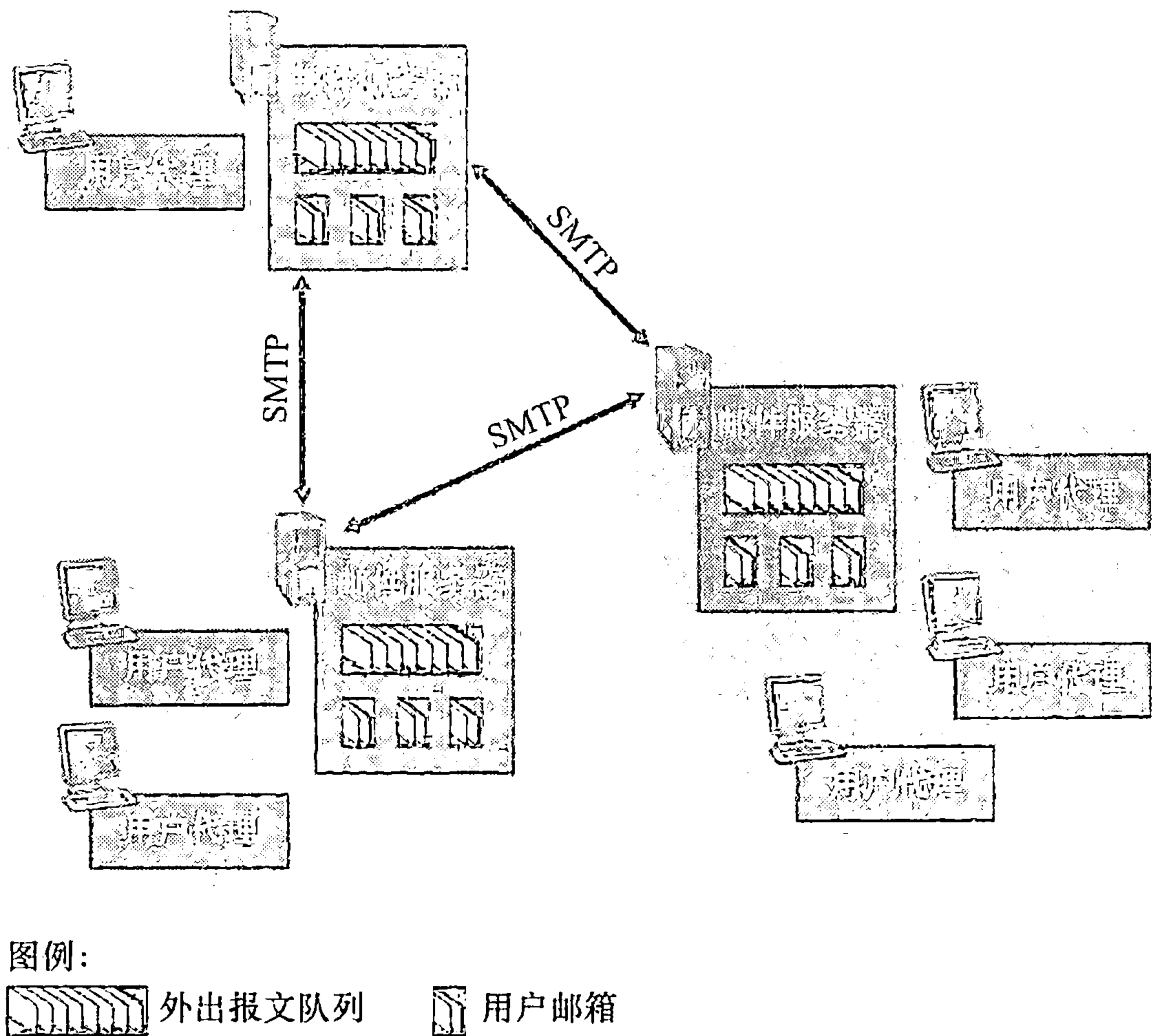


图 2-16 因特网电子邮件系统的总体描述

邮件服务器形成了电子邮件体系结构的核心。每个接收方 (如 Bob) 在其中的某个邮件服务器上有一个邮箱 (mailbox)。Bob 的邮箱管理和维护着发送给他的报文。一个典型的邮件发送过程是：从发送方的用户代理开始，传输到发送方的邮件服务器，再传输到接收方的邮件服务器，然后在这里被分发到接收方的邮箱中。当 Bob 要在他的邮箱中读取该报文时，包含他邮箱的邮件服务器 (使用用户名和口令) 来鉴别 Bob。Alice 的邮箱也必

须能处理 Bob 的邮件服务器的故障。如果 Alice 的服务器不能将邮件交付给 Bob 的服务器，Alice 的邮件服务器在一个报文队列（message queue）中保持该报文并在以后尝试再次发送。通常每 30 分钟左右进行一次尝试；如果几天后仍不能成功，服务器就删除该报文并以电子邮件的形式通知发送方（Alice）。

SMTP 是因特网电子邮件中主要的应用层协议。它使用 TCP 可靠数据传输服务，从发送方的邮件服务器向接收方的邮件服务器发送邮件。像大多数应用层协议一样，SMTP 也有两个部分：运行在发送方邮件服务器的客户端和运行在接收方邮件服务器的服务器端。每台邮件服务器上既运行 SMTP 的客户端也运行 SMTP 的服务器端。当一个邮件服务器向其他邮件服务器发送邮件时，它就表现为 SMTP 的客户；当邮件服务器从其他邮件服务器上接收邮件时，它就表现为一个 SMTP 的服务器。

2.4.1 SMTP

RFC 5321 给出了 SMTP 的定义。SMTP 是因特网电子邮件应用的核心。如前所述，SMTP 用于从发送方的邮件服务器发送报文到接收方的邮件服务器。SMTP 问世的时间比 HTTP 要长得多（初始的 SMTP 协议的 RFC 可追溯到 1982 年，而 SMTP 在此之前很长一段时间就已经出现了）。尽管电子邮件应用在因特网上的独特地位可以证明 SMTP 有着众多非常出色的性质，但它所具有的某种陈旧特征表明它仍然是一种继承的技术。例如，它限制所有邮件报文的体部分（不只是其首部）只能采用简单的 7 比特 ASCII 表示。在 20 世纪 80 年代早期，这种限制是明智的，因为当时传输能力不足，没有人会通过电子邮件发送大的附件或是大的图片、声音或者视频文件。然而，在今天的多媒体时代，7 位 ASCII 的限制的确有点痛苦，即在用 SMTP 传送邮件之前，需要将二进制多媒体数据编码为 ASCII 码，并且在使用 SMTP 传输后要求将相应的 ASCII 码邮件解码还原为多媒体数据。2.2 节讲过，使用 HTTP 传送前不需要将多媒体数据编码为 ASCII 码。

为了描述 SMTP 的基本操作，我们观察一种常见的情景。假设 Alice 想给 Bob 发送一封简单的 ASCII 报文。

- Alice 调用她的邮件代理程序并提供 Bob 的邮件地址（例如 bob@ someschool. edu），撰写报文，然后指示用户代理发送该报文。
- Alice 的用户代理把报文发给她的邮件服务器，在那里该报文被放在报文队列中。
- 运行在 Alice 的邮件服务器上的 SMTP 客户端发现了报文队列中的这个报文，它就创建一个到运行在 Bob 的邮件服务器上的 SMTP 服务器的 TCP 连接。
- 在经过一些初始 SMTP 握手后，SMTP 客户通过该 TCP 连接发送 Alice 的报文。
- 在 Bob 的邮件服务器上，SMTP 的服务器端接收该报文。Bob 的邮件服务器然后将该报文放入 Bob 的邮箱中。
- 在 Bob 方便的时候，他调用用户代理阅读该报文。

图 2-17 总结了上述这个情况。

观察到下述现象是重要的：SMTP 一般不使用中间邮件服务器发送邮件，即使这两个邮件服务器位于地球的两端也是这样。假设 Alice 的邮件服务器在中国香港，而 Bob 的服务器在美国圣路易斯，那么这个 TCP 连接也是从香港服务器到圣路易斯服务器之间的直接相连。特别是，如果 Bob 的邮件服务器没有开机，该报文会保留在 Alice 的邮件服务器上并等待进行新的尝试，这意味着邮件并不在中间的某个邮件服务器存留。

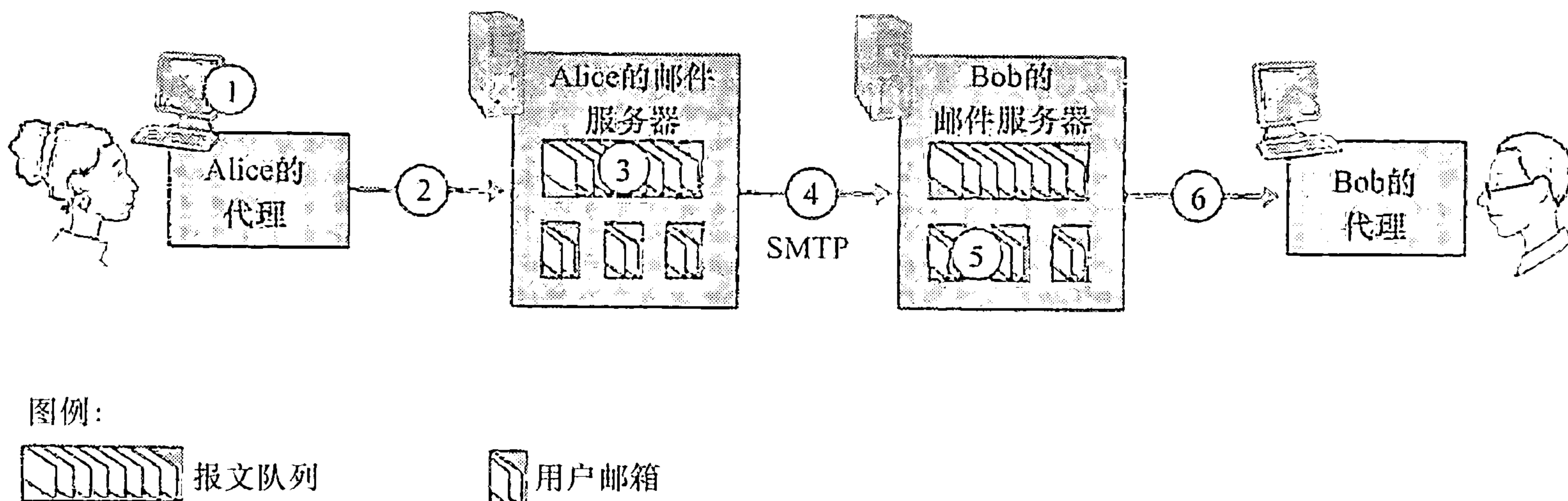


图 2-17 Alice 向 Bob 发送一条报文

我们现在仔细观察一下，SMTP 是如何将一个报文从发送邮件服务器传送到接收邮件服务器的。我们将看到，SMTP 与人类面对面交往的行为方式有许多类似性。首先，客户 SMTP（运行在发送邮件服务器上）在 25 号端口建立一个到服务器 SMTP（运行在接收邮件服务器上）的 TCP 连接。如果服务器没有开机，客户会在稍后继续尝试连接。一旦连接建立，服务器和客户执行某些应用层的握手，就像人们在互相交流前先进行自我介绍一样。SMTP 的客户和服务在传输信息前先相互介绍。在 SMTP 握手的阶段，SMTP 客户指示发送方的邮件地址（产生报文的那个人）和接收方的邮件地址。一旦该 SMTP 客户和服务彼此介绍之后，客户发送该报文。SMTP 能依赖 TCP 提供的可靠数据传输无差错地将邮件投递到接收服务器。该客户如果有另外的报文要发送到该服务器，就在该相同的 TCP 连接上重复这种处理；否则，它指示 TCP 关闭连接。

接下来我们分析一个在 SMTP 客户（C）和 SMTP 服务器（S）之间交换报文脚本的例子。客户的主机名为 `crepes.fr`，服务器的主机名为 `hamburger.edu`。以 C：开头的 ASCII 码文本行正是客户交给其 TCP 套接字的那些行，以 S：开头的 ASCII 码则是服务器发送给其 TCP 套接字的那些行。一旦创建了 TCP 连接，就开始了下列过程。

```
S: 220 hamburger.edu
C: HELO crepes.fr
S: 250 Hello crepes.fr, pleased to meet you
C: MAIL FROM: <alice@crepes.fr>
S: 250 alice@crepes.fr ... Sender ok
C: RCPT TO: <bob@hamburger.edu>
S: 250 bob@hamburger.edu ... Recipient ok
C: DATA
S: 354 Enter mail, end with "." on a line by itself
C: Do you like ketchup?
C: How about pickles?
C: .
S: 250 Message accepted for delivery
C: QUIT
S: 221 hamburger.edu closing connection
```

在上例中，客户从邮件服务器 `crepes.fr` 向邮件服务器 `hamburger.edu` 发送了一个报文（“Do you like ketchup? How about pickles?”）。作为对话的一部分，该客户发送了 5 条命令：HELO（是 HELLO 的缩写）、MAIL FROM、RCPT TO、DATA 以及 QUIT。这些命令都是自解释的。该客户通过发送一个只包含一个句点的行，向服务器指示该报文结束了。（按照 ASCII 码的表示方法，每个报文以 CRLF.CRLF 结束，其中的 CR 和 LF 分别表示回车和换行。）服务器对每条命令做出回答，其中每个回答含有一个回答码和一些（可选

的) 英文解释。我们在这里指出 SMTP 用的是持续连接: 如果发送邮件服务器有几个报文发往同一个接收邮件服务器, 它可以通过同一个 TCP 连接发送这些所有的报文。对每个报文, 该客户用一个新的 MAIL FROM: crepes.fr 开始, 用一个独立的句点指示该邮件的结束, 并且仅当所有邮件发送完后才发送 QUIT。

我们强烈推荐你使用 Telnet 与一个 SMTP 服务器进行一次直接对话。使用的命令是

```
telnet serverName 25
```

其中 serverName 是本地邮件服务器的名称。当你这么做时, 就直接在本地主机与邮件服务器之间建立一个 TCP 连接。输完上述命令后, 你立即会从该服务器收到 220 回答。接下来, 在适当的时机发出 HELO、MAIL FROM、RCPT TO、DATA、CRLF. CRLF 以及 QUIT 等 SMTP 命令。强烈推荐你做本章后面的编程作业 3。在该作业中, 你将在 SMTP 的客户端实现一个简单的用户代理, 它允许你经本地邮件服务器向任意的接收方发送电子邮件报文。

2.4.2 与 HTTP 的对比

我们简要地比较一下 SMTP 和 HTTP。这两个协议都用于从一台主机向另一台主机传送文件: HTTP 从 Web 服务器向 Web 客户 (通常是一个浏览器) 传送文件 (也称为对象); SMTP 从一个邮件服务器向另一个邮件服务器传送文件 (即电子邮件报文)。当进行文件传送时, 持续的 HTTP 和 SMTP 都使用持续连接。因此, 这两个协议有一些共同特征。然而, 两者之间也有一些重要的区别。首先, HTTP 主要是一个拉协议 (pull protocol), 即在方便的时候, 某些人在 Web 服务器上装载信息, 用户使用 HTTP 从该服务器拉取这些信息。特别是 TCP 连接是由想接收文件的机器发起的。另一方面, SMTP 基本上是一个推协议 (push protocol), 即发送邮件服务器把文件推向接收邮件服务器。特别是, 这个 TCP 连接是由要发送该文件的机器发起的。

第二个区别就是我们前面间接地提到过的, SMTP 要求每个报文 (包括它们的体) 使用 7 比特 ASCII 码格式。如果某报文包含了非 7 比特 ASCII 字符 (如具有重音的法文字符) 或二进制数据 (如图形文件), 则该报文必须按照 7 比特 ASCII 码进行编码。HTTP 数据则不受这种限制。

第三个重要区别是如何处理一个既包含文本又包含图形 (也可能是其他媒体类型) 的文档。如我们在 2.2 节知道的那样, HTTP 把每个对象封装到它自己的 HTTP 响应报文中, 而 SMTP 则把所有报文对象放在一个报文之中。

2.4.3 邮件报文格式和 MIME

当 Alice 给 Bob 写一封邮寄时间很长的普通信件时, 她可能要在信的上部包含各种各样的环境首部信息, 如 Bob 的地址、她自己的回复地址以及日期等。同样, 当一个人给另一个人发送电子邮件时, 一个包含环境信息的首部位于报文体前面。这些环境信息包括在一系列首部行中, 这些行由 RFC 5322 定义。首部行和该报文的体用空行 (即回车换行) 进行分隔。RFC 5322 定义了邮件首部行和它们的语义解释的精确格式。如同 HTTP 协议, 每个首部行包含了可读的文本, 是由关键词后跟冒号及其值组成的。某些关键词是必需的, 另一些则是可选的。每个首部必须含有一个 From: 首部行和一个 To: 首部行; 一个首部也许包含一个 Subject: 首部行以及其他可选的首部行。注意到下列事实是重要的: 这

些首部行不同于我们在 2.4.1 节所学到的 SMTP 命令（即使那里也包含了某些相同的词汇，如 from 和 to）。那节中的命令是 SMTP 握手协议的一部分；本节中研究的首部行则是邮件报文自身的一部分。

一个典型的报文首部看起来如下：

```
From: alice@crepes.fr
To: bob@hamburger.edu
Subject: Searching for the meaning of life.
```

在报文首部之后，紧接着一个空白行，然后是以 ASCII 格式表示的报文体。你应当用 Telnet 向邮件服务器发送包含一些首部行的报文，包括 Subject：首部行。为此，输入命令 telnet serverName 25，如在 2.4.1 节中讨论的那样。

2.4.4 邮件访问协议

一旦 SMTP 将邮件报文从 Alice 的邮件服务器交付给 Bob 的邮件服务器，该报文就被放入了 Bob 的邮箱中。在此讨论中，我们按惯例假定 Bob 是通过登录到服务器主机，并直接在该主机上运行一个邮件阅读程序来阅读他的邮件的。直到 20 世纪 90 年代早期，这都是一种标准方式。而在今天，邮件访问使用了一种客户-服务器体系结构，即典型的用户通过在用户端系统上运行的客户程序来阅读电子邮件，这里的端系统可能是办公室的 PC、便携机或者是智能手机。通过在本地主机上运行邮件客户程序，用户享受一系列丰富的特性，包括查看多媒体报文和附件的能力。

假设 Bob（接收方）在其本地 PC 上运行用户代理程序，考虑在他的本地 PC 上也放置一个邮件服务器是自然而然的事。在这种情况下，Alice 的邮件服务器就能直接与 Bob 的 PC 进行对话了。然而这种方法会有一个问题。前面讲过邮件服务器管理用户的邮箱，并且运行 SMTP 的客户端和服务端。如果 Bob 的邮件服务器位于他的 PC 上，那么为了能够及时接收可能在任何时候到达的新邮件，他的 PC 必须总是不间断地运行着并一直保持在线。这对于大多数因特网用户而言是不现实的。相反，典型的用户通常在本地图 PC 上运行一个用户代理程序，而它访问存储在总是保持开机的共享邮件服务器上的邮箱。该邮件服务器与其他用户共享，并且通常由用户的 ISP 进行维护（如大学或公司）。

现在我们考虑当从 Alice 向 Bob 发送一个电子邮件报文时所取的路径。我们刚才已经知道，在沿着该路径的某些点上，电子邮件报文存放在 Bob 的邮件服务器上。通过让 Alice 的用户代理直接向 Bob 的邮件服务器发送报文，就能够做到这一点。这能够由 SMTP 来完成：实际上，SMTP 被设计成将电子邮件从一台主机推到另一台主机。然而，通常 Alice 的用户代理和 Bob 的邮件服务器之间并没有一个直接的 SMTP 对话。相反，如图 2-18 所示，Alice 的用户代理用 SMTP 将电子邮件报文推入她的邮件服务器，接着她的邮件服务器（作为一个 SMTP 客户）再用 SMTP 将该邮件中继到 Bob 的邮件服务器。为什么该过程要分成两步呢？主要是因为不通过 Alice 的邮件服务器进行中继，Alice 的用户代理将没有任何办法到达一个不可达的目的地接收服务器。通过首先将邮件存放在自己的邮件服务器中，Alice 的邮件服务器可以重复地尝试向 Bob 的邮件服务器发送该报文，如每 30 分钟一次，直到 Bob 的邮件服务器变得运行为止。（并且如果 Alice 的邮件服务器关机，她则能向系统管理员进行申告！）SMTP RFC 文档定义了如何使用 SMTP 命令经过多个 SMTP 服务器进行报文中继。

但是这里仍然有一个疏漏的环节！像 Bob 这样的接收方，是如何通过运行其本地 PC

上的用户代理，获得位于他的某 ISP 的邮件服务器上的邮件呢？值得注意的是 Bob 的用户代理不能使用 SMTP 取回报文，因为取报文是一个拉操作，而 SMTP 协议是一个推协议。通过引入一个特殊的邮件访问协议来解决这个难题，该协议将 Bob 邮件服务器上的报文传送给他的本地 PC。目前有一些流行的邮件访问协议，包括第三版的邮局协议（Post Office Protocol—Version 3，POP3）、因特网邮件访问协议（Internet Mail Access Protocol，IMAP）以及 HTTP。

图 2-18 总结了应用于因特网电子邮件的一些协议：SMTP 用来将邮件从发送方的邮件服务器传输到接收方的邮件服务器；SMTP 也用来将邮件从发送方的用户代理传送到发送方的邮件服务器。如 POP3 这样的邮件访问协议用来将邮件从接收方的邮件服务器传送到接收方的用户代理。

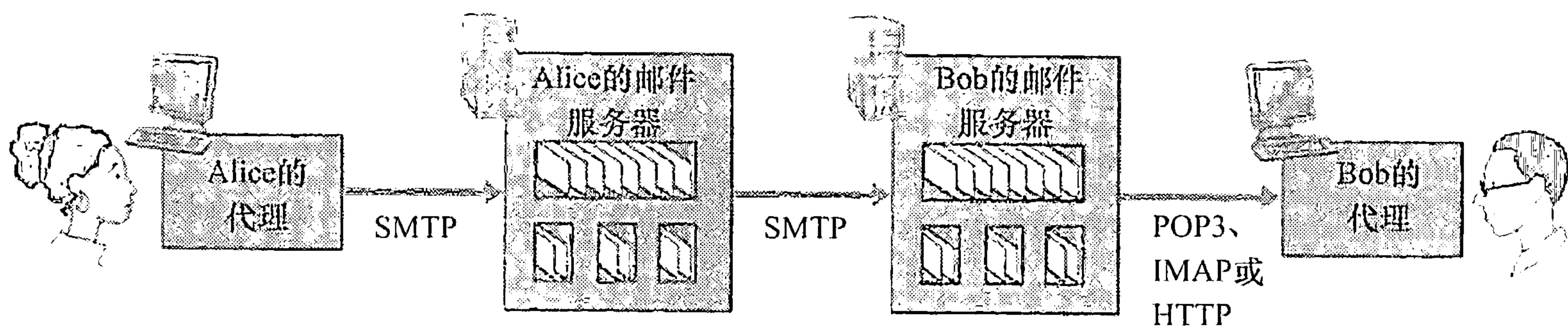


图 2-18 电子邮件协议及其通信实体

1. POP3

POP3 是一个极为简单的邮件访问协议，由 RFC 1939 进行定义。文档 RFC 1939 简短且可读性强。因为该协议非常简单，故其功能相当有限。当用户代理（客户）打开了一个到邮件服务器（服务器）端口 110 上的 TCP 连接后，POP3 就开始工作了。随着建立 TCP 连接，POP3 按照三个阶段进行工作：特许（authorization）、事务处理以及更新。在第一个阶段即特许阶段，用户代理发送（以明文形式）用户名和口令以鉴别用户。在第二个阶段即事务处理阶段，用户代理取回报文；同时在这个阶段用户代理还能进行如下操作，对报文做删除标记，取消报文删除标记，以及获取邮件的统计信息。在第三个阶段即更新阶段，它出现在客户发出了 quit 命令之后，目的是结束该 POP3 会话；这时，该邮件服务器删除那些被标记为删除的报文。

在 POP3 的事务处理过程中，用户代理发出一些命令，服务器对每个命令做出回答。回答可能有两种：+OK（有时后面还跟有服务器到客户的数据），被服务器用来指示前面的命令是正常的；-ERR，被服务器用来指示前面的命令出现了某些差错。

特许阶段有两个主要的命令：user <user name> 和 pass <password>。为了举例说明这两个命令，我们建议你直接用 Telnet 登录到 POP3 服务器的 110 端口，然后发出这两个命令。假设邮件服务器的名字为 mailServer，那么你将看到类似的过程：

```
telnet mailServer 110
+OK POP3 server ready
user bob
+OK
pass hungry
+OK user successfully logged on
```

如果你的命令拼写错了，该 POP3 服务器将返回一个 -ERR 报文。

现在我们来看一下事务处理过程。使用 POP3 的用户代理通常被用户配置为“下载并

删除”或者“下载并保留”方式。POP3 用户代理发出的命令序列取决于用户代理程序被配置为这两种工作方式的哪一种。使用下载并删除方式，用户代理发出 list、retr 和 dele 命令。举例来说，假设用户在他（她）的邮箱里有两个报文。在下面的对话中，C：（代表客户）是用户代理，S：（代表服务器）是邮件服务器。事务处理过程将类似于如下过程：

```
C: list
S: 1 498
S: 2 912
S: .
C: retr 1
S: (blah blah ...
S: .....
S: .....blah)
S: .
C: dele 1
C: retr 2
S: (blah blah ...
S: .....
S: .....blah)
S: .
C: dele 2
C: quit
S: +OK POP3 server signing off
```

用户代理首先请求邮件服务器列出所有存储的报文的长度。接着用户代理从邮件服务器取回并删除每封邮件。注意在特许阶段以后，用户代理仅使用四个命令：list、retr、dele 和 quit，这些命令的语法定义在 RFC 1939 中。在处理 quit 命令后，POP3 服务器进入更新阶段，从用户的邮箱中删除邮件 1 和 2。

使用下载并删除方式存在的问题是，邮件接收方 Bob 可能是移动的，可能希望从多个不同的机器访问他的邮件报文，如从办公室的 PC、家里的 PC 或他的便携机来访问邮件。下载并删除方式将对 Bob 的邮件报文根据这 3 台机器进行划分，特别是如果 Bob 最先是在他办公室的 PC 上收取了一条邮件，那么晚上当他在家里时，通过他的便携机将不能再收取该邮件。使用下载并保留方式，用户代理下载某邮件后，该邮件仍保留在邮件服务器上。这时，Bob 就能通过不同的机器重新读取这些邮件；他能在工作时收取一封报文，而在工作回家后再次访问它。

在用户代理与邮件服务器之间的 POP3 会话期间，该 POP3 服务器保留了一些状态信息；特别是记录了哪些用户报文被标记为删除了。然而，POP3 服务器并不在 POP3 会话过程中携带状态信息。会话中不包括状态信息大大简化了 POP3 服务的实现。

2. IMAP

使用 POP3 访问时，一旦 Bob 将邮件下载到本地主机后，他就能建立邮件文件夹，并将下载的邮件放入该文件夹中。然后 Bob 可以删除报文，在文件夹之间移动报文，并查询报文（通过发送方的名字或报文主题）。但是这种文件夹和报文存放在本地主机上的方式，会给移动用户带来问题，因为他更喜欢使用一个在远程服务器上的层次文件夹，这样他可以从任何一台机器上对所有报文进行访问。POP3 不可能做到这一点，POP3 协议没有给用户提供任何创建远程文件夹并为报文指派文件夹的方法。

为了解决这个或其他一些问题，由 RFC 3501 定义的因特网邮件访问协议（IMAP）应运而生。和 POP3 一样，IMAP 是一个邮件访问协议，但是它比 POP3 具有更多的特色，不过也比 POP3 复杂得多。（因此客户和服务端端的实现也都复杂得多。）

IMAP 服务器把每个报文与一个文件夹联系起来；当报文第一次到达服务器时，它与收件人的 INBOX 文件夹相关联。收件人则能够把邮件移到一个新的、用户创建的文件夹中，阅读邮件，删除邮件等。IMAP 协议为用户提供了创建文件夹以及将邮件从一个文件夹移动到另一个文件夹的命令。IMAP 还为用户提供了在远程文件夹中查询邮件的命令，按指定条件去查询匹配的邮件。值得注意的是，与 POP3 不同，IMAP 服务器维护了 IMAP 会话的用户状态信息，例如，文件夹的名字以及哪些报文与哪些文件夹相关联。

IMAP 的另一个重要特性是它具有允许用户代理获取报文组件的命令。例如，一个用户代理可以只读取一个报文的报文首部，或只是一个多部分 MIME 报文的一部分。当用户代理和其邮件服务器之间使用低带宽连接的时候，这个特性非常有用（如一个低速调制解调器链路）。使用这种低带宽连接时，用户可能并不想取回他邮箱中的所有邮件，尤其要避免可能包含如音频或视频片断的大邮件。

3. 基于 Web 的电子邮件

今天越来越多的用户使用他们的 Web 浏览器收发电子邮件。20 世纪 90 年代中期 Hotmail 引入了基于 Web 的接入。今天，谷歌、雅虎以及几乎所有重要的大学或者公司也提供了基于 Web 的电子邮件。使用这种服务，用户代理就是普通的浏览器，用户和他远程邮箱之间的通信则通过 HTTP 进行。当一个收件人（如 Bob），想从他的邮箱中访问一个报文时，该电子邮件报文从 Bob 的邮件服务器发送到他的浏览器，使用的是 HTTP 而不是 POP3 或者 IMAP 协议。当发件人（如 Alice）要发送一封电子邮件报文时，该电子邮件报文从 Alice 的浏览器发送到她的邮件服务器，使用的是 HTTP 而不是 SMTP。然而，Alice 的邮件服务器在与其他的邮件服务器之间发送和接收邮件时，仍然使用的是 SMTP。

2.5 DNS：因特网的目录服务

人类能以很多方式来标识。例如，我们能够通过出生证书上的名字来标识；能够通过社会保险号码来标识；也能够通过驾驶执照上的号码来标识。尽管这些标识办法都可以用来识别一个人，但是在特定环境下，某种识别方法可能比另一种方法更为适合。例如，IRS（美国的一个声名狼藉的税务征收机构）的计算机更喜欢使用定长的社会保险号码而不是出生证书上的姓名。另一方面，普通人乐于使用更好记的出生证书上的姓名而不是社会保险号码。（毫无疑问，你能想象人们之间以这种方式说话吗？如“你好，我叫 132-67-9875。请找一下我的丈夫 178-87-1146”。）

因特网上的主机和人类一样，可以使用多种方式进行标识。主机的一种标识方法是用它的主机名（hostname），如 cnn.com、www.yahoo.com、gaia.cs.umass.edu 以及 cis.poly.edu 等，这些名字便于记忆也乐于被人们接受。然而，主机名几乎没有提供（即使有也很少）关于主机在因特网中位置的信息。（一个名为 www.eurecom.fr 的主机以国家码 .fr 结束，告诉我们该主机很可能在法国，仅此而已。）况且，因为主机名可能由不定长的字母数字组成，路由器难以处理。由于这些原因，主机也可以使用所谓 IP 地址（IP address）进行标识。

我们将在第 4 章更为详细地讨论 IP 地址，但现在简略地介绍一下还是有必要的。一个 IP 地址由 4 个字节组成，并有着严格的层次结构。例如 121.7.106.83 这样一个 IP 地址，其中的每个字节都被句点分隔开来，表示了 0~255 的十进制数字。我们说 IP 地址具

有层次结构，是因为当我们从左至右扫描它时，我们会得到越来越具体的关于主机位于因特网何处的信息（即在众多网络的哪个网络里）。类似地，当我们从下向上查看邮政地址时，我们能够获得该地址位于何处的越来越具体的信息。

2.5.1 DNS 提供的服务

我们刚刚看到了识别主机有两种方式，通过主机名或者 IP 地址。人们喜欢便于记忆的主机名标识方式，而路由器则喜欢定长的、有着层次结构的 IP 地址。为了折衷这些不同的偏好，我们需要一种能进行主机名到 IP 地址转换的目录服务。这就是域名系统（Domain Name System, DNS）的主要任务。DNS 是：①一个由分层的 DNS 服务器（DNS server）实现的分布式数据库；②一个使得主机能够查询分布式数据库的应用层协议。DNS 服务器通常是运行 BIND（Berkeley Internet Name Domain）软件 [BIND 2012] 的 UNIX 机器。DNS 协议运行在 UDP 之上，使用 53 号端口。

实践原则

DNS：通过客户 - 服务器模式提供的重要网络功能

与 HTTP、FTP 和 SMTP 协议一样，DNS 协议是应用层协议，其原因在于：①使用客户 - 服务器模式运行在通信的端系统之间；②在通信的端系统之间通过下面的端到端运输协议来传送 DNS 报文。然而，在其他意义上，DNS 的作用非常不同于 Web 应用、文件传输应用以及电子邮件应用。与这些应用程序不同之处在于，DNS 不是一个直接和用户打交道的应用。相反，DNS 是为因特网上的用户应用程序以及其他软件提供一种核心功能，即将主机名转换为其背后的 IP 地址。我们在 1.2 节就提到，因特网体系结构的复杂性大多数位于网络的“边缘”。DNS 通过采用了位于网络边缘的客户和服务，实现了关键的名字到地址转换功能，它还是这种设计原理的另一个范例。

DNS 通常是由其他应用层协议所使用的，包括 HTTP、SMTP 和 FTP，将用户提供的主机名解析为 IP 地址。举一个例子，考虑当某个用户主机上的一个浏览器（即一个 HTTP 客户）请求 URL `www.someschool.edu/index.html` 页面时会发生什么现象。为了使用户的主机能够将一个 HTTP 请求报文发送到 Web 服务器 `www.someschool.edu`，该用户主机必须获得 `www.someschool.edu` 的 IP 地址。其做法如下。

- 同一台用户主机上运行着 DNS 应用的客户端。
- 浏览器从上述 URL 中抽取出主机名 `www.someschool.edu`，并将这台主机名传给 DNS 应用的客户端。
- DNS 客户向 DNS 服务器发送一个包含主机名的请求。
- DNS 客户最终会收到一份回答报文，其中含有对应于该主机名的 IP 地址。
- 一旦浏览器接收到来自 DNS 的该 IP 地址，它能够向位于该 IP 地址 80 端口的 HTTP 服务器进程发起一个 TCP 连接。

从这个例子中，我们可以看到 DNS 给使用它的因特网应用带来了额外的时延，有时还相当可观。幸运的是，如我们下面讨论的那样，想获得的 IP 地址通常就缓存在一个“附近的” DNS 服务器中，这有助于减少 DNS 的网络流量和 DNS 的平均时延。

除了进行主机名到 IP 地址的转换外，DNS 还提供了一些重要的服务：

- **主机别名 (host aliasing)**。有着复杂主机名的主机能拥有一个或者多个别名。例如，一台名为 `relay1.west-coast.enterprise.com` 的主机，可能还有两个别名为 `enterprise.com` 和 `www.enterprise.com`。在这种情况下，`relay1.west-coast.enterprise.com` 也称为**规范主机名 (canonical hostname)**。主机别名（当存在时）比主机规范名更加容易记忆。应用程序可以调用 DNS 来获得主机别名对应的规范主机名以及主机的 IP 地址。
- **邮件服务器别名 (mail server aliasing)**。显而易见，人们也非常希望电子邮件地址好记忆。例如，如果 Bob 在 Hotmail 上有一个账户，Bob 的邮件地址就像 `bob@hotmail.com` 这样简单。然而，Hotmail 邮件服务器的主机名可能更为复杂，不像 `hotmail.com` 那样简单好记（例如，规范主机名可能像 `relay1.west-coast.hotmail.com` 那样）。电子邮件应用程序可以调用 DNS，对提供的邮件服务器别名进行解析，以获得该主机的规范主机名及其 IP 地址。事实上，MX 记录（参见后面）允许一个公司的邮件服务器和 Web 服务器使用相同（别名化的）的主机名；例如，一个公司的 Web 服务器和邮件服务器都能叫做 `enterprise.com`。
- **负载分配 (load distribution)**。DNS 也用于在冗余的服务器（如冗余的 Web 服务器等）之间进行负载分配。繁忙的站点（如 `cnn.com`）被冗余分布在多台服务器上，每台服务器均运行在不同的端系统上，每个都有着不同的 IP 地址。由于这些冗余的 Web 服务器，一个 IP 地址集合因此与同一个规范主机名相联系。DNS 数据库中存储着这些 IP 地址集合。当客户对映射到某地址集合的名字发出一个 DNS 请求时，该服务器用 IP 地址的整个集合进行响应，但在每个回答中循环这些地址次序。因为客户通常总是向 IP 地址排在最前面的服务器发送 HTTP 请求报文，所以 DNS 就在所有这些冗余的 Web 服务器之间循环分配了负载。DNS 的循环同样可以用于邮件服务器，因此，多个邮件服务器可以具有相同的别名。一些内容分发公司如 Akamai 也以更加复杂的方式使用 DNS [Dilley 2002]，以提供 Web 内容分发（参见第 7 章）。

DNS 由 RFC 1034 和 RFC 1035 定义，并且在几个附加的 RFC 中进行了更新。DNS 是一个复杂的系统，我们在这里只是就其运行的主要方面进行学习。感兴趣的读者可以参考这些 RFC 文档和 Albitz 和 Liu 写的书 [Albitz 1993]；亦可参阅文章 [Mockapetris 1998] 和 [Mockapetris 2005]，其中 [Mockapetris 1998] 是回顾性的文章，它提供了 DNS 组成和工作原理的精细的描述。

2.5.2 DNS 工作机理概述

下面给出一个 DNS 工作过程的总体概括，我们的讨论将集中在主机名到 IP 地址转换服务方面。

假设运行在用户主机上的某些应用程序（如 Web 浏览器或邮件阅读器）需要将主机名转换为 IP 地址。这些应用程序将调用 DNS 的客户端，并指明需要被转换的主机名（在很多基于 UNIX 的机器上，应用程序为了执行这种转换需要调用函数 `gethostbyname()`）。用户主机上的 DNS 接收到后，向网络中发送一个 DNS 查询报文。所有的 DNS 请求和回答报文使用 UDP 数据报经端口 53 发送。经过若干毫秒到若干秒的时延后，用户主机上的

DNS 接收到一个提供所希望映射的 DNS 回答报文。这个映射结果则被传递到调用 DNS 的应用程序。因此，从用户主机上调用应用程序的角度看，DNS 是一个提供简单、直接的转换服务的黑盒子。但事实上，实现这个服务的黑盒子非常复杂，它由分布于全球的大量 DNS 服务器以及定义了 DNS 服务器与查询主机通信方式的应用层协议组成。

DNS 的一种简单设计是在因特网上只使用一个 DNS 服务器，该服务器包含所有的映射。在这种集中式设计中，客户直接将所有查询直接发往单一的 DNS 服务器，同时该 DNS 服务器直接对所有的查询客户做出响应。尽管这种设计的简单性非常具有吸引力，但它不适用于当今的因特网，因为因特网有着数量巨大（并持续增长）的主机。这种集中式设计的问题包括：

- **单点故障**（a single point of failure）。如果该 DNS 服务器崩溃，整个因特网随之瘫痪！
- **通信容量**（traffic volume）。单个 DNS 服务器不得不处理所有的 DNS 查询（用于为上亿台主机产生的所有 HTTP 请求报文和电子邮件报文服务）。
- **远距离的集中式数据库**（distant centralized database）。单个 DNS 服务器不可能“邻近”所有查询客户。如果我们将单台 DNS 服务器放在纽约市，那么所有来自澳大利亚的查询必须传播到地球的另一边，中间也许还要经过低速和拥塞的链路。这将导致严重的时延。
- **维护**（maintenance）。单个 DNS 服务器将不得不为所有的因特网主机保留记录。这不仅将使这个中央数据库非常庞大，而且它还不得不为解决每个新添加的主机而频繁更新。

总的来说，在单一 DNS 服务器上运行集中式数据库完全没有可扩展能力。因此，DNS 采用了分布式的设计方案。事实上，DNS 是一个在因特网上实现分布式数据库的精彩范例。

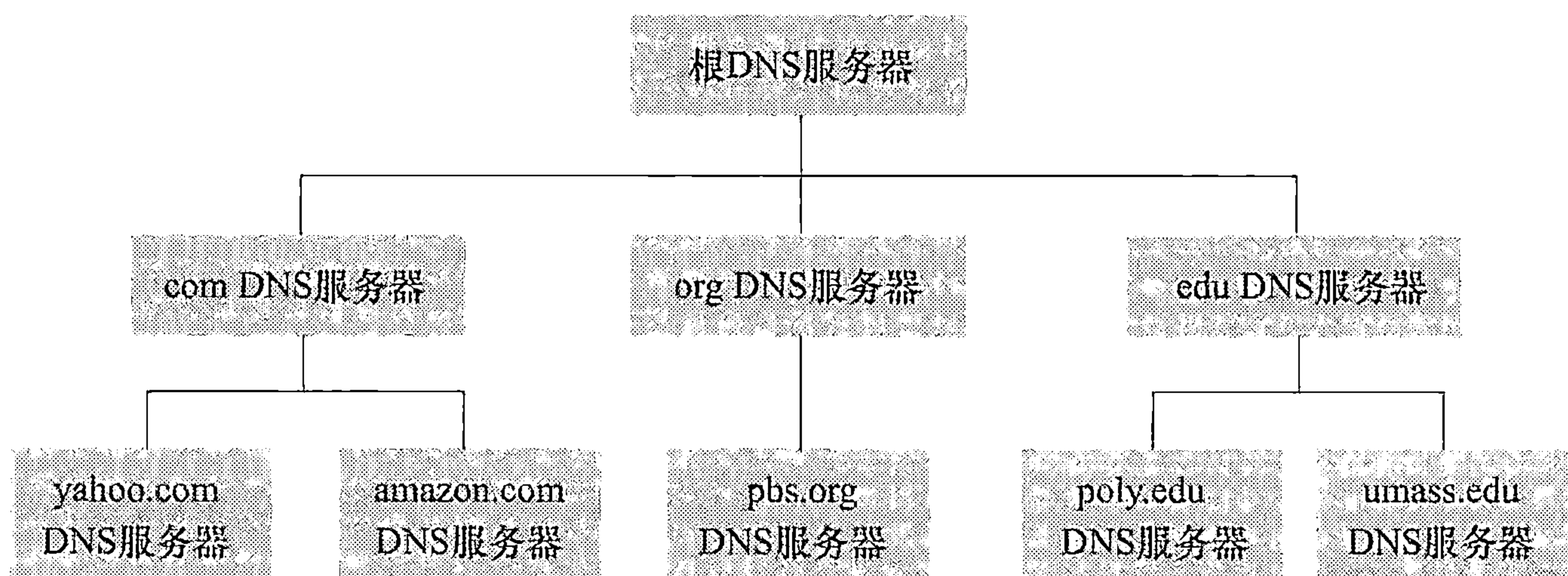


图 2-19 DNS 服务器的部分层次结构

1. 分布式、层次数据库

为了处理扩展性问题，DNS 使用了大量的 DNS 服务器，它们以层次方式组织，并且分布在全世界范围内。没有一台 DNS 服务器拥有因特网上所有主机的映射。相反，该映射分布在所有的 DNS 服务器上。大致说来，有 3 种类型的 DNS 服务器：根 DNS 服务器、顶级域（Top-Level Domain, TLD）DNS 服务器和权威 DNS 服务器。这些服务器以图 2-19 中所示的层次结构组织起来。为了理解这 3 种类型的 DNS 服务器交互的方式，假定一个 DNS 客户要决定主机名 `www.amazon.com` 的 IP 地址。粗略说来，将发生下列事件。客户首

先与根服务器之一联系，它将返回顶级域名 com 的 TLD 服务器的 IP 地址。该客户则与这些 TLD 服务器之一联系，它将为 amazon.com 返回权威服务器的 IP 地址。最后，该客户与 amazon.com 权威服务器之一联系，它为主机名 www.amazon.com 返回其 IP 地址。我们将很快更为详细地考察 DNS 查找过程。不过我们先仔细看一下这 3 种类型的 DNS 服务器。

- **根 DNS 服务器。**在因特网上有 13 个根 DNS 服务器（标号为 A 到 M），它们中的大部分位于北美洲。图 2-20 中显示的是一张 2012 年的根 DNS 服务器分布图；通过 [Root-servers 2012] 可查看当前可用的根 DNS 服务器列表。尽管我们将这 13 个根 DNS 服务器中的每个都视为单个的服务器，但每台“服务器”实际上是一个冗余服务器的网络，以提供安全性和可靠性。到了 2011 年秋季，共有 247 个根服务器。
- **顶级域（DNS）服务器。**这些服务器负责顶级域名如 com、org、net、edu 和 gov，以及所有国家的顶级域名如 uk、fr、ca 和 jp。Verisign Global Registry Services 公司维护 com 顶级域的 TLD 服务器；Educause 公司维护 edu 顶级域的 TLD 服务器。所有顶级域的列表参见 [IANA TLD 2012]。
- **权威 DNS 服务器。**在因特网上具有公共可访问主机（如 Web 服务器和邮件服务器）的每个组织机构必须提供公共可访问的 DNS 记录，这些记录将这些主机的名字映射为 IP 地址。一个组织机构的权威 DNS 服务器收藏了这些 DNS 记录。一个组织机构能够选择实现它自己的权威 DNS 服务器以保存这些记录；另一种方法是，该组织能够支付费用，让这些记录存储在某个服务提供商的一个权威 DNS 服务器中。多数大学和大公司实现和维护它们自己基本和辅助（备份）的权威 DNS 服务器。

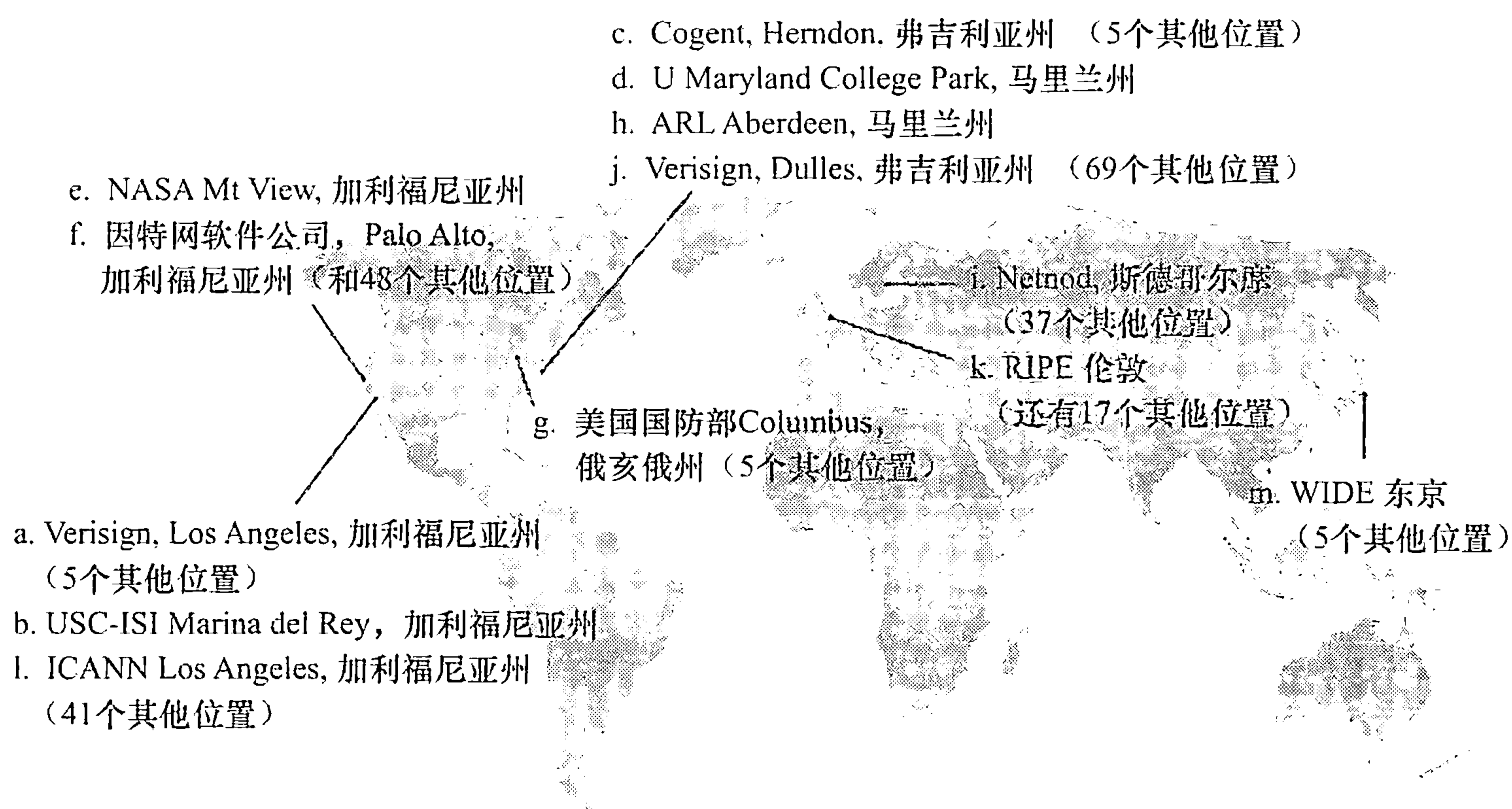


图 2-20 2012 年的 DNS 根服务器（名称、组织和位置）

根、TLD 和权威 DNS 服务器都处在该 DNS 服务器的层次结构中，如图 2-19 中所示。还有另一类重要的 DNS，称为本地 DNS 服务器（local DNS server）。一个本地 DNS 服务器严格说来并不属于该服务器的层次结构，但它对 DNS 层次结构是重要的。每个 ISP（如一个大学、一个系、一个公司或一个居民区的 ISP）都有一台本地 DNS 服务器（也叫默认名字服务器）。当主机与某个 ISP 连接时，该 ISP 提供一台主机的 IP 地址，该主机具有一台

或多台其本地 DNS 服务器的 IP 地址（通常通过 DHCP，将在第 4 章中讨论）。通过访问 Windows 或 UNIX 的网络状态窗口，能够容易地确定你本地 DNS 服务器的 IP 地址。主机的本地 DNS 服务器通常“邻近”本主机。对某机构 ISP 而言，本地 DNS 服务器可能就和主机在同一个局域网中；对于某居民区 ISP 来说，本地 DNS 服务器通常与主机相隔不超过几台路由器。当主机发出 DNS 请求时，该请求被发往本地 DNS 服务器，它起着代理的作用，并将该请求转发到 DNS 服务器层次结构中，我们下面将更为详细地讨论。

我们来看一个简单的例子，假设主机 `cis.poly.edu` 想知道主机 `gaia.cs.umass.edu` 的 IP 地址。同时假设理工大学（Polytechnic）的本地 DNS 服务器为 `dns.poly.edu`，并且 `gaia.cs.umass.edu` 的权威 DNS 服务器为 `dns.umass.edu`。如图 2-21 所示，主机 `cis.poly.edu` 首先向它的本地 DNS 服务器 `dns.poly.edu` 发送一个 DNS 查询报文。该查询报文含有被转换的主机名 `gaia.cs.umass.edu`。本地 DNS 服务器将该报文转发到根 DNS 服务器。该根 DNS 服务器注意到其 `edu` 前缀并向本地 DNS 服务器返回负责 `edu` 的 TLD 的 IP 地址列表。该本地 DNS 服务器则再次向这些 TLD 服务器之一发送查询报文。该 TLD 服务器注意到 `umass.edu` 前缀，并用权威 DNS 服务器的 IP 地址进行响应，该权威 DNS 服务器是负责马萨诸塞大学的 `dns.umass.edu`。最后，本地 DNS 服务器直接向 `dns.umass.edu` 重发查询报文，`dns.umass.edu` 用 `gaia.cs.umass.edu` 的 IP 地址进行响应。注意到在本例中，为了获得一台主机名的映射，共发送了 8 份 DNS 报文：4 份查询报文和 4 份回答报文！我们将很快看到利用 DNS 缓存减少这种查询流量的方法。

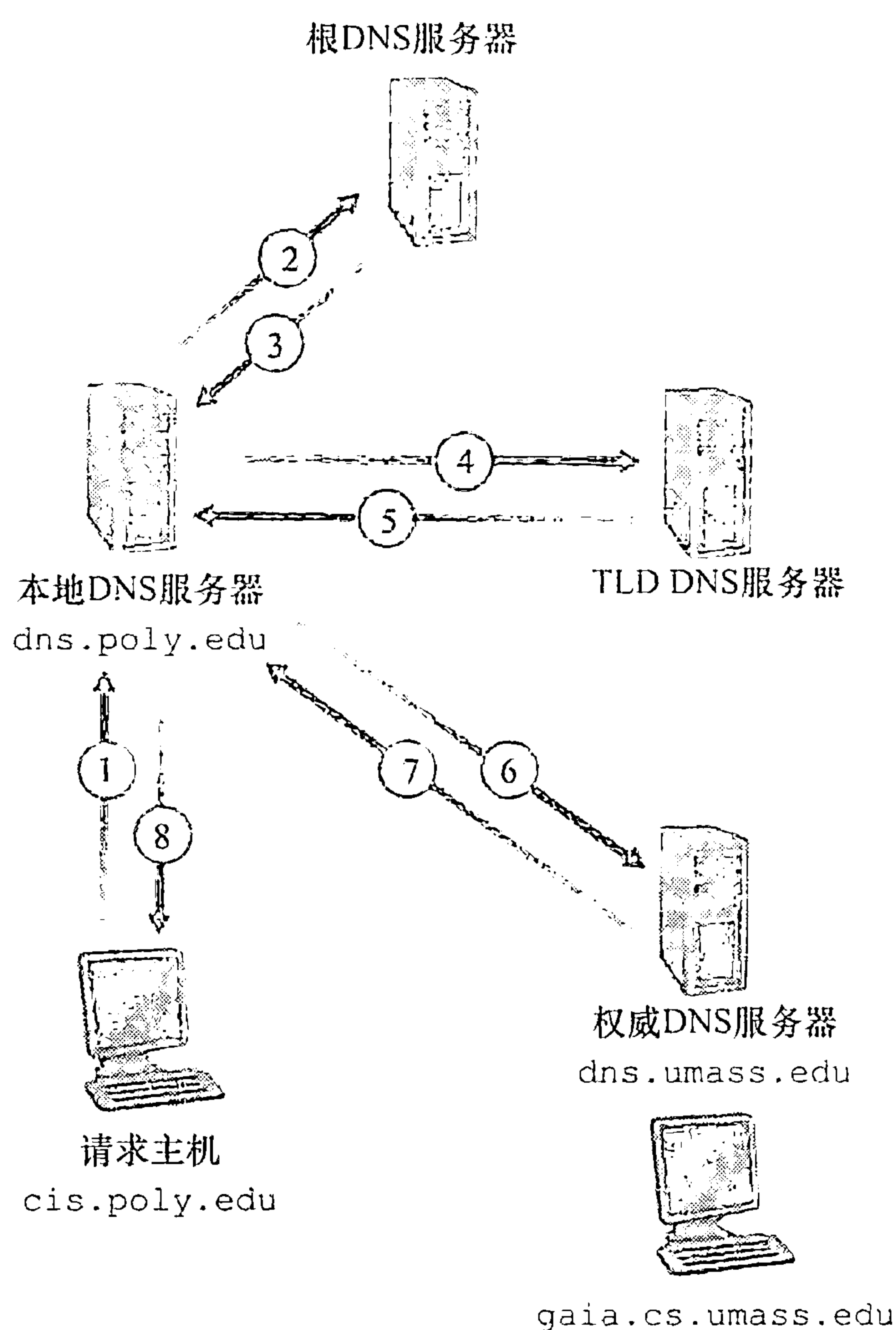


图 2-21 各种 DNS 服务器的交互

我们前面的例子假设了 TLD 服务器知道用于主机的权威 DNS 服务器的 IP 地址。一般而言，这种假设并不总是正确的。相反，TLD 服务器只是知道中间的某个 DNS 服务器，该中间 DNS 服务器依次才能知道用于该主机的权威 DNS 服务器。例如，再次假设马萨诸塞大学有一台用于本大学的 DNS 服务器，它称为 `dns.umass.edu`。同时假设该大学的每个系都有自己的 DNS 服务器，每个系的 DNS 服务器是本系所有主机的权威服务器。在这种情况下，当中间 DNS 服务器 `dns.umass.edu` 收到了对某主机的请求时，该主机名是以 `cs.umass.edu` 结尾，它向 `dns.poly.edu` 返回 `dns.cs.umass.edu` 的 IP 地址，后者是所有以 `cs.umass.edu` 结尾的主机的权威服务器。本地 DNS 服务器 `dns.poly.edu` 则向权威 DNS 服务器发送查询，该权威 DNS 服务器将请求的映射发送给本地 DNS 服务器，该本地服务器依次向请求主机返回该映射。在这个例子中，共发送了 10 份 DNS 报文！

图 2-21 所示的例子利用了递归查询（recursive query）和迭代查询（iterative query）。从

cis.poly.edu 到 dns.poly.edu 发出的查询是递归查询，因为该查询请求 dns.poly.edu 以自己的名义获得该映射。而后继的 3 个查询是迭代查询，因为所有的回答都是直接返回给 dns.poly.edu。从理论上讲，任何 DNS 查询既可以是迭代的也能是递归的。例如，图 2-22 显示了一条 DNS 查询链，其中的所有查询都是递归的。实践中，查询通常遵循图 2-21 中的模式。从请求主机到本地 DNS 服务器的查询是递归的，其余的查询是迭代的。

2. DNS 缓存

至此我们的讨论还没有涉及 DNS 系统的一个非常重要特色：DNS 缓存（DNS caching）。实际上，为了改善时延性能并减少在因特网上到处传输的 DNS 报文数量，DNS 广泛使用了缓存技术。DNS 缓存的原理非常简单。在一个请求链中，当某 DNS 服务器接收一个 DNS 回答（例如，包含主机名到 IP 地址的映射）时，它能够将该回答中的信息缓存在本地存储器中。例如，在图 2-21 中，每当本地 DNS 服务器 dns.poly.edu 从某个 DNS 服务器接收到一个回答，它能够缓存包含在该回答中的任何信息。如果在 DNS 服务器中缓存了一台主机名/IP 地址对，另一个对相同主机名的查询到达该 DNS 服务器时，该 DNS 服务器就能够提

供所要求的 IP 地址，即使它不是该主机名的权威服务器。由于主机和主机名与 IP 地址间的映射并不是永久的，DNS 服务器在一段时间后（通常设置为两天）将丢弃缓存的信息。

举一个例子，假定主机 apricot.poly.edu 向 dns.poly.edu 查询主机名 cnn.com 的 IP 地址。此后，假定过了几个小时，Polytechnic 理工大学的另外一台主机如 kiwi.poly.edu 也向 dns.poly.edu 查询相同的主机名。因为有了缓存，该本地 DNS 服务器可以立即返回 cnn.com 的 IP 地址，而不必查询任何其他 DNS 服务器。本地 DNS 服务器也能够缓存 TLD 服务器的 IP 地址，因而允许本地 DNS 绕过查询链中的根 DNS 服务器（这经常发生）。

2.5.3 DNS 记录 and 报文

共同实现 DNS 分布式数据库的所有 DNS 服务器存储了资源记录（Resource Record, RR），RR 提供了主机名到 IP 地址的映射。每个 DNS 回答报文包含了一条或多条资源记录。在本小节以及后续小节中，我们概要地介绍 DNS 资源记录 and 报文；更详细的信息可以在 [Albitz 1993] 或有关 DNS 的 RFC 文档 [RFC 1034; RFC 1035] 中找到。

资源记录是一个包含了下列字段的 4 元组：

(Name, Value, Type, TTL)

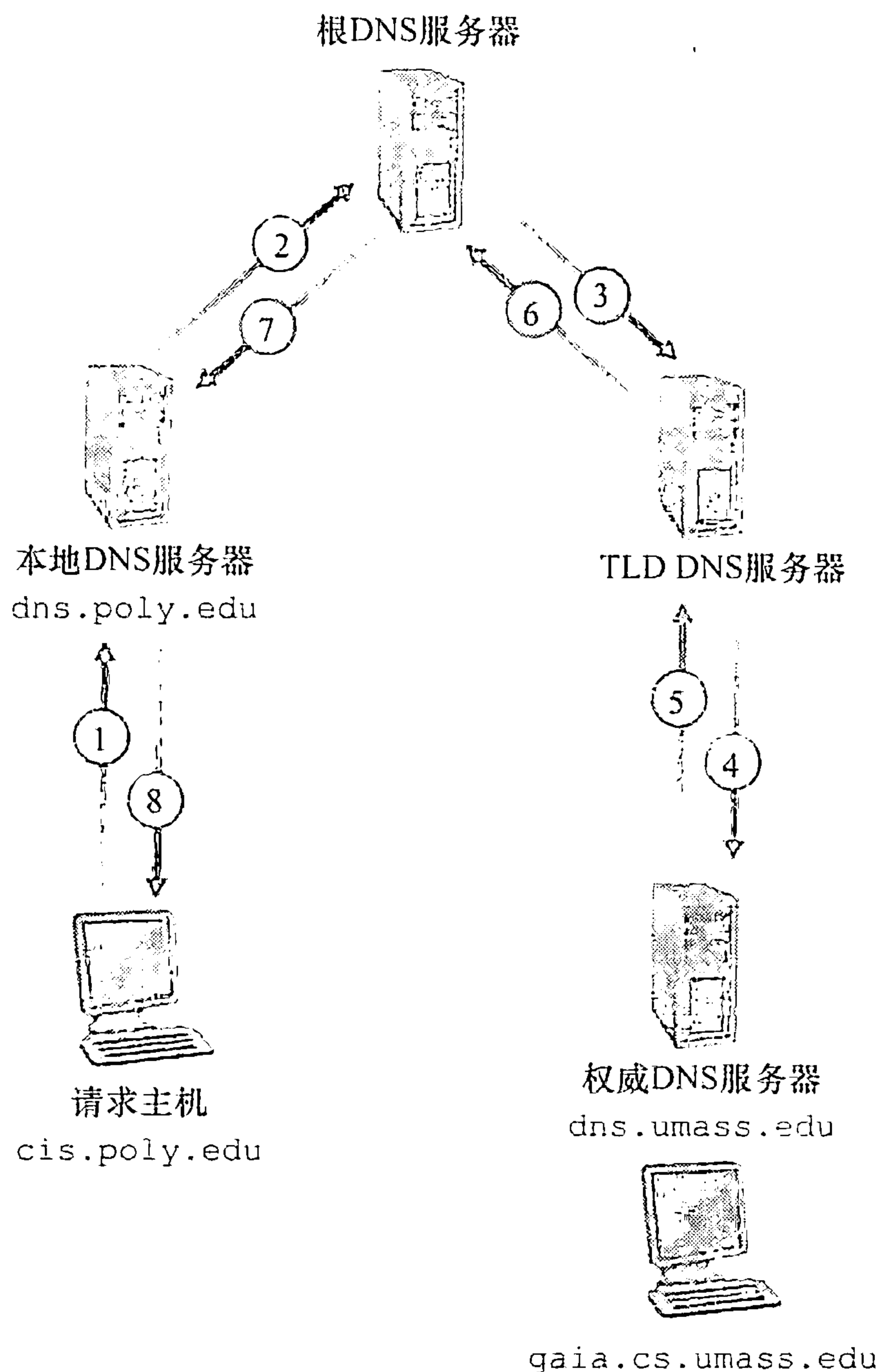


图 2-22 DNS 中的递归查询

TTL 是该记录的生存时间，它决定了资源记录应当从缓存中删除的时间。在下面给出的记录例子中，我们忽略掉 TTL 字段。Name 和 Value 的值取决于 Type：

- 如果 Type = A，则 Name 是主机名，Value 是该主机名对应的 IP 地址。因此，一条类型为 A 的资源记录提供了标准的主机名到 IP 地址的映射。例如（relay1.bar.foo.com, 145.37.93.126, A）就是一条类型 A 记录。
- 如果 Type = NS，则 Name 是个域（如 foo.com），而 Value 是个知道如何获得该域中主机 IP 地址的权威 DNS 服务器的主机名。这个记录用于沿着查询链来路由 DNS 查询。例如（foo.com, dns.foo.com, NS）就是一条类型为 NS 的记录。
- 如果 Type = CNAME，则 Value 是别名为 Name 的主机对应的规范主机名。该记录能够向查询的主机提供一个主机名对应的规范主机名，例如（foo.com, relay1.bar.foo.com, CNAME）就是一条 CNAME 类型的记录。
- 如果 Type = MX，则 Value 是个别名为 Name 的邮件服务器的规范主机名。举例来说，（foo.com, mail.bar.foo.com, MX）就是一条 MX 记录。MX 记录允许邮件服务器主机名具有简单的别名。值得注意的是，通过使用 MX 记录，一个公司的邮件服务器和其他服务器（如它的 Web 服务器）可以使用相同的别名。为了获得邮件服务器的规范主机名，DNS 客户应当请求一条 MX 记录；而为了获得其他服务器的规范主机名，DNS 客户应当请求 CNAME 记录。

如果一台 DNS 服务器是用于某特定主机名的权威 DNS 服务器，那么该 DNS 服务器会有一条包含该主机名的类型 A 记录（即使该 DNS 服务器不是其权威 DNS 服务器，它也可能在缓存中包含有一条类型 A 记录）。如果服务器不是用于某主机名的权威服务器，那么该服务器将包含一条类型 NS 记录，该记录对应于包含主机名的域；它还将包括一条类型 A 记录，该记录提供了在 NS 记录的 Value 字段中的 DNS 服务器的 IP 地址。举例来说，假设一台 edu TLD 服务器不是主机 gaia.cs.umass.edu 的权威 DNS 服务器，则该服务器将包含一条包括主机 cs.umass.edu 的域记录，如（umass.edu, dns.umass.edu, NS）；该 edu TLD 服务器还将包含一条类型 A 记录，如（dns.umass.edu, 128.119.40.111, A），该记录将名字 dns.umass.edu 映射为一个 IP 地址。

1. DNS 报文

在本节前面，我们提到了 DNS 查询和回答报文。DNS 只有这两种报文，并且，查询和回答报文有着相同的格式，如图 2-23 所示。DNS 报文中各字段的语义如下：

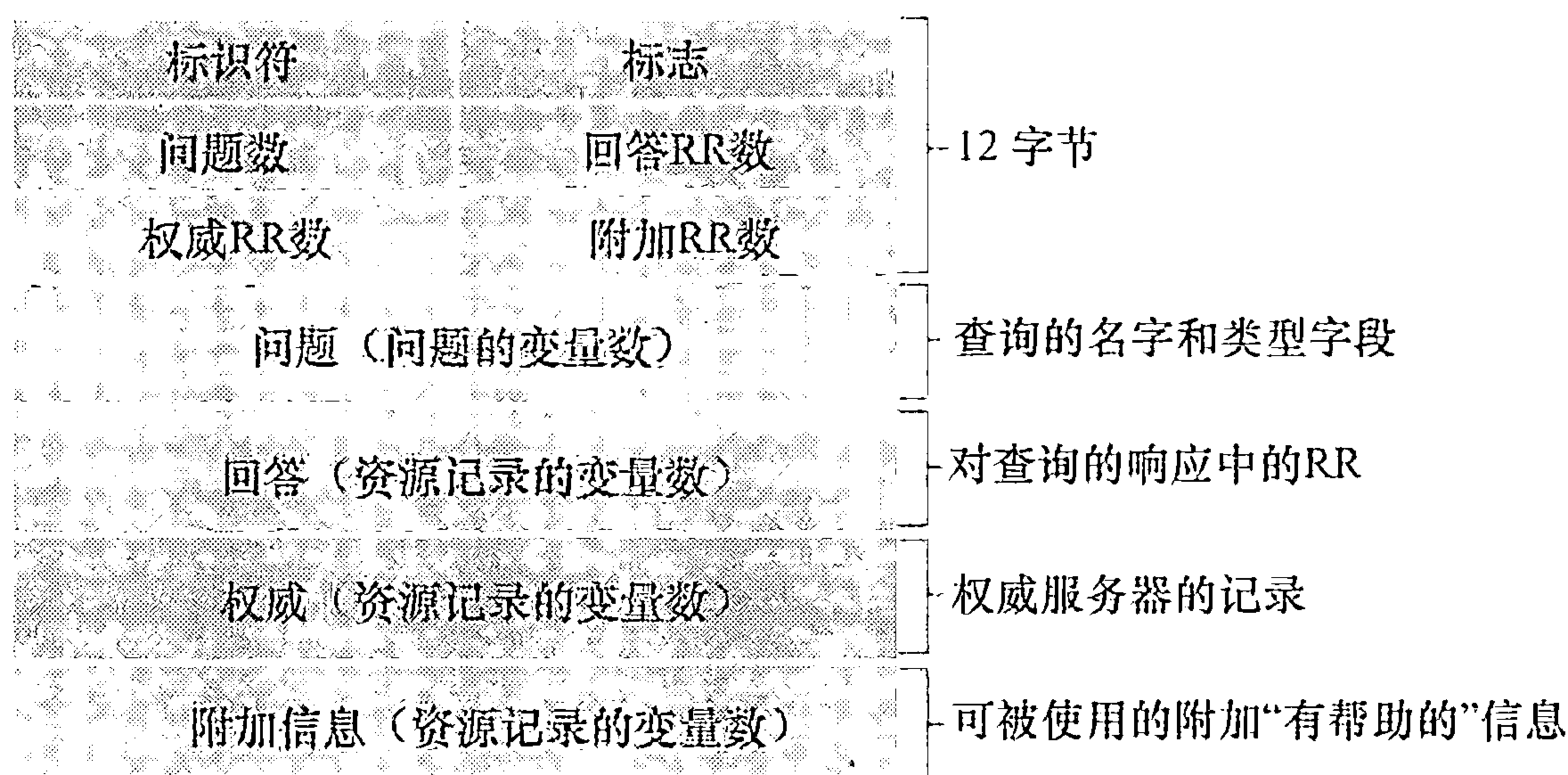


图 2-23 DNS 报文格式

- 前 12 个字节是首部区域，其中有几个字段。第一个字段（标识符）是一个 16 比特的数，用于标识该查询。这个标识符会被复制到对查询的回答报文中，以便让客户用它来匹配发送的请求和接收到的回答。标志字段中含有若干标志。1 比特的“查询/回答”标志位指出报文是查询报文（0）还是回答报文（1）。当某 DNS 服务器是所请求名字的权威 DNS 服务器时，1 比特的“权威的”标志位被置在回答报文中。如果客户（主机或者 DNS 服务器）在该 DNS 服务器没有某记录时希望它执行递归查询，将设置 1 比特的“希望递归”标志位。如果该 DNS 服务器支持递归查询，在它的回答报文中会对 1 比特的“递归可用”标志位置位。在该首部中，还有 4 个有关数量的字段，这些字段指出了在首部后的 4 类数据区域出现的数量。
- 问题区域包含着正在进行的查询信息。该区域包括：①名字字段，指出正在被查询的主机名字；②类型字段，它指出有关该名字的正被询问的问题类型，例如主机地址是与一个名字相关联（类型 A）还是与某个名字的邮件服务器相关联（类型 MX）。
- 在来自 DNS 服务器的回答中，回答区域包含了对最初请求的资源的资源记录。前面讲过每个资源记录中有 Type（如 A、NS、CNAME 和 MX）字段、Value 字段和 TTL 字段。在回答报文的回答区域中可以包含多条 RR，因此一个主机名能够有多个 IP 地址（例如，就像本节前面讨论的冗余 Web 服务器）。
- 权威区域包含了其他权威服务器的记录。
- 附加区域包含了其他有帮助的记录。例如，对于一个 MX 请求的回答报文的回答区域包含了一条资源记录，该记录提供了邮件服务器的规范主机名。该附加区域包含一个类型 A 记录，该记录提供了用于该邮件服务器的规范主机名的 IP 地址。

你愿意从正在工作的主机直接向某些 DNS 服务器发送一个 DNS 查询报文吗？使用 nslookup 程序（nslookup program）能够容易地做到这一点，对于多数 Windows 和 UNIX 平台，nslookup 程序是可用的。例如，从一台 Windows 主机打开命令提示符界面，直接键入“nslookup”即可调用该 nslookup 程序。在调用 nslookup 后，你能够向任何 DNS 服务器（根、TLD 或权威）发送 DNS 查询。在接收到来自 DNS 服务器的回答后，nslookup 将显示包括在该回答中的记录（以人可读的格式）。从你自己的主机运行 nslookup 还有一种方法，即访问允许你远程应用 nslookup 的许多 Web 站点之一（在一个搜索引擎中键入“nslookup”就能够得到这些站点中的一个）。本章最后的 DNS Wireshark 实验将使你更为详细地研究 DNS。

2. 在 DNS 数据库中插入记录

上面的讨论只是关注如何从 DNS 数据库中取数据。你可能想知道这些数据最初是怎么进入数据库中的。我们在一个特定的例子中看看这是如何完成的。假定你刚刚创建一个称为网络乌托邦（Network Utopia）的令人兴奋的新创业公司。你必定要做的第一件事是在注册登记机构注册域名 networkutopia.com。注册登记机构（registrar）是一个商业实体，它验证该域名的唯一性，将该域名输入 DNS 数据库（如下面所讨论的那样），对提供的服务收取少量费用。1999 年前，唯一的注册登记机构是 Network Solution，它独家经营对于 com、net 和 org 域名的注册。但是现在有许多注册登记机构竞争客户，因特网名字和地址分配机构（Internet Corporation for Assigned Names and Numbers, ICANN）向各种注册登记机构授权。在 <http://www.internic.net> 上可以找到授权的注册登记机构的列表。

当你向某些注册登记机构注册域名 `networkutopia.com` 时，需要向该机构提供你的基本和辅助权威 DNS 服务器的名字和 IP 地址。假定该名字和 IP 地址是 `dns1.networkutopia.com` 和 `dns2.networkutopia.com` 及 `212.212.212.1` 和 `212.212.212.2`。对这两个权威 DNS 服务器的每一个，该注册登记机构确保将一个类型 NS 和一个类型 A 的记录输入 TLD `com` 服务器。特别是对于用于 `networkutopia.com` 的基本权威服务器，该注册登记机构将下列两条资源记录插入该 DNS 系统中：

```
(networkutopia.com, dns1.networkutopia.com, NS)
```

```
(dns1.networkutopia.com, 212.212.212.1, A)
```

你还必须确保用于 Web 服务器 `www.networkutopia.com` 的类型 A 资源记录和用于邮件服务器 `mail.networkutopia.com` 的类型 MX 资源记录被输入你的权威 DNS 服务器中。（直到最近，每个 DNS 服务器中的内容都是静态配置的，例如来自系统管理员创建的配置文件。最近，在 DNS 协议中添加了一个更新（UPDATE）选项，允许通过 DNS 报文对数据库中的内容进行动态添加或者删除。[RFC 2136] 和 [RFC 3007] 定义了 DNS 动态更新。）

一旦完成所有这些步骤，人们将能够访问你的 Web 站点，并向你公司的雇员发送电子邮件。我们通过验证该说法的正确性来总结 DNS 的讨论。这种验证也有助于充实我们已经学到的 DNS 知识。假定在澳大利亚的 Alice 要观看 `www.networkutopia.com` 的 Web 页面。如前面所讨论，她的主机将首先向其本地 DNS 服务器发送请求。该本地服务器接着则联系一个 TLD `com` 服务器。（如果 TLD `com` 服务器的地址没有被缓存，该本地 DNS 服务器也将必须与根 DNS 服务器相联系。）该 TLD 服务器包含前面列出的类型 NS 和类型 A 资源记录，因为注册登记机构将这些资源记录插入所有的 TLD `com` 服务器。该 TLD `com` 服务器向 Alice 的本地 DNS 服务器发送一个回答，该回答包含了这两条资源记录。该本地 DNS 服务器则向 `212.212.212.1` 发送一个 DNS 查询，请求对应于 `www.networkutopia.com` 的类型 A 记录。该记录提供了所希望的 Web 服务器的 IP 地址，如 `212.212.71.4`，本地 DNS 服务器将该地址回传给 Alice 的主机。Alice 的浏览器此时能够向主机 `212.212.71.4` 发起一个 TCP 连接，并在该连接上发送一个 HTTP 请求。当一个人在网上冲浪时，有比满足眼球更多的事情在进行！

关注安全性

DNS 脆弱性

我们已经看到 DNS 是因特网基础设施的一个至关重要的组件，对于包括 Web、电子邮件等的许多重要的服务，没有它都不能正常工作。因此，我们自然要问，DNS 能够被怎样攻击呢？DNS 是一个易受攻击的目标吗？它是将会被淘汰的服务吗？大多数因特网应用会随同它一起无法工作吗？

想到的第一种针对 DNS 服务的攻击是分布式拒绝服务（DDoS）带宽洪泛攻击（参见 1.6 节）。例如，某攻击者能够试图向每个 DNS 根服务器发送大量的分组，使得大多数合法 DNS 请求得不到回答。这种对 DNS 根服务器的 DDoS 大规模攻击实际发生在 2002 年 10 月 21 日。在这次攻击中，该攻击者利用了一个僵尸网络向 13 个 DNS 根服务器中的每个都发送了大批的 ICMP ping 报文。（第 4 章中讨论了 ICMP 报文。此时，知道 ICMP

分组是特殊类型的 IP 数据报就可以了。) 幸运的是, 这种大规模攻击所带来的损害很小, 对用户的因特网体验几乎没有或根本没有影响。攻击者的确成功地将大量的分组指向了根服务器, 但许多 DNS 根服务器受到了分组过滤器的保护, 配置的分组过滤器阻挡了所有指向根服务器的 ICMP ping 报文。这些被保护的服务器因此未受伤害并且与平常一样发挥着作用。此外, 大多数本地 DNS 服务器缓存了顶级域名服务器的 IP 地址, 使得这些请求过程通常绕过了 DNS 根服务器。

对 DNS 的潜在更为有效的 DDoS 攻击将是向顶级域名服务器 (例如向所有处理 .com 域的顶级域名服务器) 发送大量的 DNS 请求。过滤指向 DNS 服务器的 DNS 请求将更为困难, 并且顶级域名服务器不像根服务器那样容易绕过。但是这种攻击的严重性通过本地 DNS 服务器中的缓存技术可将部分地被缓解。

DNS 能够潜在地以其他方式被攻击。在中间人攻击中, 攻击者截获来自主机的请求并返回伪造的回答。在 DNS 毒害攻击中, 攻击者向一台 DNS 服务器发送伪造的回答, 诱使服务器在它的缓存中接收伪造的记录。这些攻击中的任一种, 都能够将满怀信任的 Web 用户重定向到攻击者的 Web 站点。然而, 这些攻击难以实现, 因为它们要求截获分组或扼制住服务器 [Skoudis 2006]。

另一种重要的 DNS 攻击本质上并不是一种对 DNS 服务的攻击, 而是充分利用 DNS 基础设施来对目标主机发起 DDoS 攻击 (例如, 你所在大学的邮件服务器)。在这种攻击中, 攻击者向许多权威 DNS 服务器发送 DNS 请求, 每个请求具有目标主机的假冒源地址。这些 DNS 服务器则直接向目标主机发送它们的回答。如果这些请求能够精心制作成下述方式的话, 即响应比请求 (字节数) 大得多 (所谓放大), 则攻击者不必自行产生大量的流量就有可能淹没目标主机。这种利用 DNS 的反射攻击至今为止只取得了有限的成功 [Mirkovic 2005]。

总而言之, DNS 自身已经显示了对抗攻击的令人惊讶的健壮性。至今为止, 还没有一个攻击已经成功地妨碍了 DNS 服务。已经有了成功的反射攻击; 然而, 通过适当地配置 DNS 服务器, 能够处理 (和正在处理) 这些攻击。

2.6 P2P 应用

在目前为止本章中描述的应用 (包括 Web、电子邮件和 DNS) 都采用了客户 - 服务器体系结构, 极大地依赖于总是打开的基础设施服务器。2.1.1 节讲过, 使用 P2P 体系结构, 对总是打开的基础设施服务器有最小的 (或者没有) 依赖。与之相反, 成对间歇连接的主机 (称为对等方) 彼此直接通信。这些对等方并不为服务提供商所拥有, 而是受用户控制的桌面计算机和膝上计算机。

在本节中我们将研究两种不同的特别适合于 P2P 设计的应用。第一种应用是文件分发, 其中应用程序从单个源向大量的对等方分发一个文件。文件分发是开始研究 P2P 的良好起点, 因为它清晰地揭示了 P2P 体系结构的自扩展性。作为文件分发的一个特定的例子, 我们将描述流行的 BitTorrent 协议。我们将研究的第二种 P2P 应用是分布在大型对等方社区中的数据库。对于这个应用, 我们将探讨分布式散列表 (Distributed Hash Table, DHT) 的概念。

2.6.1 P2P 文件分发

我们通过考虑一个非常自然的应用来开始涉足 P2P，这种应用即从单一服务器向大量主机（称为对等方）分发一个大文件。该文件也许是一个新版的 Linux 操作系统，对于现有操作系统或应用程序的一个软件补丁，一个 MP3 音乐文件，或一个 MPEG 视频文件。在客户 - 服务器文件分发中，该服务器必须向每个对等方发送该文件的一个副本，即服务器承受了极大的负担，并且消耗了大量的服务器带宽。在 P2P 文件分发中，每个对等方能够重新分发它所有的该文件的任何部分，从而在分发过程中协助该服务器。到 2012 年止，最为流行的 P2P 文件共享协议是 BitTorrent。该应用程序最初由 Bram Cohen 所研发，现在有许多不同的独立且符合 BitTorrent 协议的 BitTorrent 客户，就像有许多符合 HTTP 协议的 Web 浏览器客户一样。在下面的小节中，我们首先研究在文件分发环境中的 P2P 体系结构的自扩展性。然后我们更为详细地描述 BitTorrent，突出它的最为重要的特性和特色。

1. P2P 体系结构的扩展性

为了将客户 - 服务器体系结构与 P2P 体系结构进行比较，阐述 P2P 的内在自扩展性，我们现在考虑一个用于两种体系结构类型的简单定量模型，将一个文件分发给一个固定集合。如图 2-24 所示，服务器和对等方使用接入链路与因特网相连。其中 u_s 表示服务器接入链路上载速率， u_i 表示第 i 对等方接入链路上载速率， d_i 表示了第 i 对等方接入链路的下载速率。还用 F 表示被分发的文件长度（以比特计）， N 表示要获得的该文件副本的对等方的数量。分发时间（distribution time）是所有 N 个对等方得到该文件的副本所需要的时间。在下面分析分发时间的过程中，我们对客户 - 服务器和 P2P 体系结构做了简化（并且一般是准确的 [Akella 2003]）的假设，即因特网核心具有足够的带宽，这意味着所有瓶颈都在网络接入链路。我们还假设服务器和客户没有参与任何其他网络应用，因此它们的所有上传和下载访问带宽能被全部用于分发该文件。

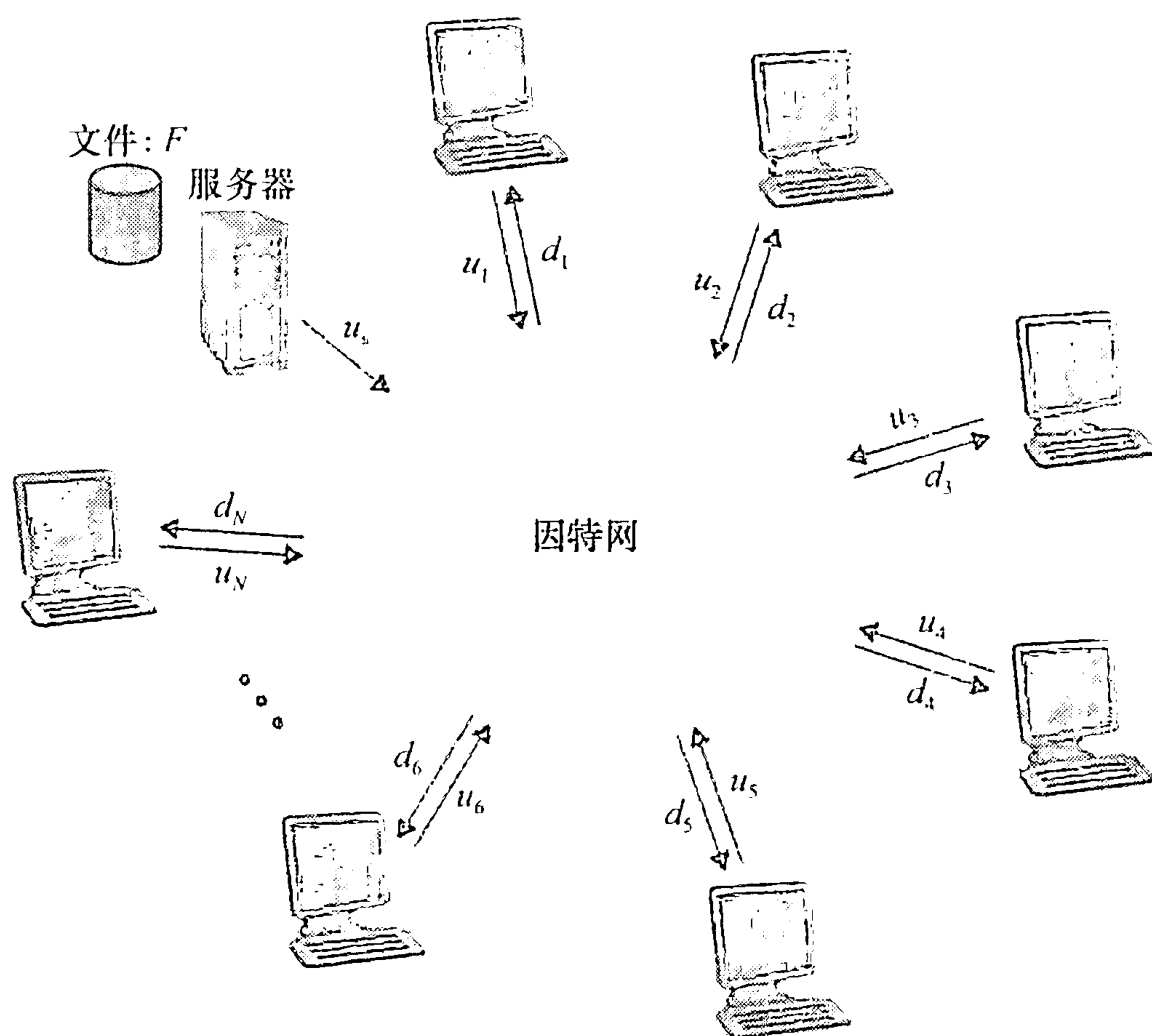


图 2-24 文件分发问题的示例图

我们首先来确定对于客户-服务器体系结构的分发时间，我们将其表示为 D_{cs} 。在客户-服务器体系结构中，没有对等方参与来帮助分发文件。我们做下列观察：

- 服务器必须向 N 个对等方的每个传输该文件的一个副本。因此该服务器必须传输 NF 比特。因为该服务器的上载速率是 u_s ，分发该文件的时间必定是至少为 NF/u_s 。
- 令 $d_{\min}$ 表示具有最小下载速率的对等方的下载速率，即 $d_{\min} = \min\{d_1, d_2, \dots, d_N\}$ 。具有最小下载速率的对等方不可能在少于 $F/d_{\min}$ 秒时间内获得该文件的所有 F 比特。因此最小分发时间至少为 $F/d_{\min}$ 。

将这两个观察放在一起，我们得到

$$D_{cs} \geq \max\left\{\frac{NF}{u_s}, \frac{F}{d_{\min}}\right\}$$

该式提供了对于客户-服务器体系结构的最小分发时间的下界。在课后习题中将请你给出服务器能够调度它的传输以便实际取得该下界的方法。因此我们取上面提供的这个下界作为实际发送时间，即

$$D_{cs} = \max\left\{\frac{NF}{u_s}, \frac{F}{d_{\min}}\right\} \quad (2-1)$$

我们从式 (2-1) 看到，对足够大的 N ，客户-服务器分发时间由 NF/u_s 确定。所以，该分发时间随着对等方 N 的数量线性地增加。因此举例来说，如果从某星期到下星期对等方的数量从 1000 增加了 1000 倍，到了 100 万，将该文件分发到所有对等方所需要的时间就要增加 1000 倍。

我们现在来对 P2P 体系结构进行简单的分析，其中每个对等方能够帮助服务器分发该文件。特别是，当一个对等方接收到某些文件数据，它能够使用自己的上载能力重新将数据分发给其他对等方。计算 P2P 体系结构的分发时间在某种程度上比计算客户-服务器体系结构的更为复杂，因为分发时间取决于每个对等方如何向其他对等方分发该文件的各个部分。无论如何，能够得到对该最小分发时间的一个简单表示式 [Kumar 2006]。至此，我们先做下列观察：

- 在分发的开始，只有服务器具有文件。为了使社区的这些对等方得到该文件，该服务器必须经其接入链路至少发送该文件的每个比特一次。因此，最小分发时间至少是 F/u_s 。（与客户-服务器方案不同，由服务器发送过一次的比特可能不必由该服务器再次发送，因为对等方在它们之间可以重新分发这些比特。）
- 与客户-服务器体系结构相同，具有最低下载速率的对等方不能够以小于 $F/d_{\min}$ 秒的分发时间获得所有 F 比特。因此最小分发时间至少为 $F/d_{\min}$ 。
- 最后，观察到系统整体的总上载能力等于服务器的上载速率加上每个单独的对等方的上载速率，即 $u_{\text{total}} = u_s + u_1 + \dots + u_N$ 。系统必须向这 N 个对等方的每个交付（上载） F 比特，因此总共交付 NF 比特。这不能以快于 u_{total} 的速率完成。因此，最小的分发时间也至少是 $NF/(u_s + u_1 + \dots + u_N)$ 。

将这三个观察放在一起，我们获得了对 P2P 的最小分发时间，表示为 D_{P2P} 。

$$D_{\text{P2P}} \geq \max\left\{\frac{F}{u_s}, \frac{F}{d_{\min}}, \frac{NF}{u_s + \sum_{i=1}^N u_i}\right\} \quad (2-2)$$

式 (2-2) 提供了对于 P2P 体系结构的最小分发时间的下界。这说明, 如果我们认为一旦每个对等方接收到一个比特就能够重分发一个比特的话, 则存在一个重新分发方案能实际取得这种下界 [Kumar 2006]。(我们将在课后习题中证明该结果的一种特情形。) 实际上, 被分发的是文件块而不是一个个比特。式 (2-2) 能够作为实际最小分发时间的很好近似。因此, 我们取由式 (2-2) 提供的下界作为实际的最小分发时间, 即

$$D_{\text{P2P}} = \max \left\{ \frac{F}{u_s}, \frac{F}{d_{\min}}, \frac{NF}{u_s + \sum_{i=1}^N u_i} \right\} \quad (2-3)$$

图 2-25 比较了客户-服务器和 P2P 体系结构的最小分发时间, 其中假定所有的对等方具有相同的上载速率 u 。在图 2-25 中, 我们已经设置了 $F/u = 1$ 小时, $u_s = 10u$, $d_{\min} \geq u_s$ 。因此, 在一个小时中一个对等方能够传输整个文件, 该服务器的传输速率是对等方上载速率的 10 倍, 并且 (为了简化起见) 对等方的下载速率被设置得足够大, 使之不会产生影响。我们从图 2-25 中看到, 对于客户-服务器体系结构, 随着对等方数量的增加, 分发时间呈线性增长并且没有界。然而, 对于 P2P 体系结构, 最小分发时间不仅总是小于客户-服务器体系结构的分发时间, 并且对于任意的对等方数量 N , 总是小于 1 小时。因此, 具有 P2P 体系结构的程序能够是自扩展的。这种扩展性的直接成因是: 对等方除了是比特的消费者外还是它们的重新分发者。

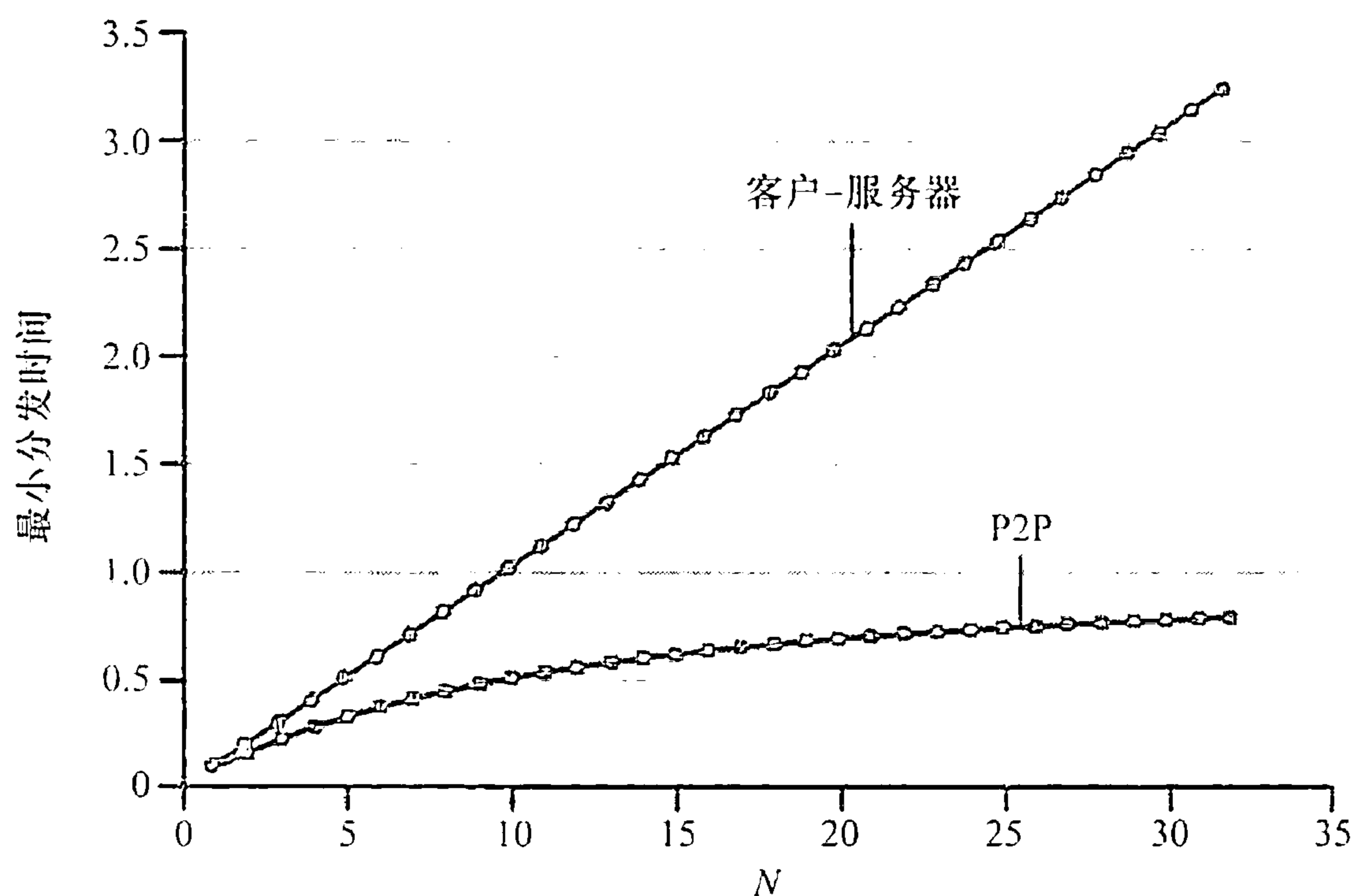


图 2-25 P2P 和客户-服务器体系结构的分发时间

2. BitTorrent

BitTorrent 是一种用于文件分发的流行 P2P 协议 [Chao 2011]。用 BitTorrent 的术语来讲, 参与一个特定文件分发的所有对等方的集合被称为一个洪流 (torrent)。在一个洪流中的对等方彼此下载等长度的文件块 (chunk), 典型的块长度为 256KB。当一个对等方首次加入一个洪流时, 它没有块。随着时间的流逝, 它累积了越来越多的块。当它下载块时, 也为其他对等方上载了多个块。一旦某对等方获得了整个文件, 它也许 (自私地) 离开洪流, 或 (大公无私地) 留在该洪流中并继续向其他对等方上载块。同时, 任何对等方可能任何时候仅具有块的子集就离开该洪流, 并在以后重新加入该洪流中。

我们现在更为仔细地观察 BitTorrent 运行的过程。因为 BitTorrent 是一个相当复杂的协